

===== FILE: .pending_commits/2025-10-11-commit-01.txt =====

- 1: feat(cache): Phase 1 - temp URL cache + page hydration
- 2:
- 3: - add _utils/file-url-cache.js with batch+TTL+multi-end resolver
- 4: - add _utils/hydrate-temp-urls.js and wire into index & post-detail
- 5: - fix index.vue async/await scope and this-binding
- 6: - make main.js \$tcb wrapper multi-end friendly (H5/App/MP)
- 7:
- 8: No cloudfunctions changed.

===== FILE: .pending_commits/2025-10-11-commit-02.txt =====

- 1: feat(cache): Phase 2 - introduce CacheManager and wire file-url-cache to namespace
- 2:
- 3: - add _utils/cache-manager.js (memory LRU + optional uniStorage persistence)
- 4: - update _utils/file-url-cache.js to read/write via CacheManager namespace('fileUrls')
- 5: - keep default persistent OFF for fileUrls to avoid cross-user mixing; can enable later
- 6: - no cloudfunctions changed; API of file-url-cache remains the same
- 7:

===== FILE: .pending_commits/2025-10-11-commit-03.txt =====

- 1: feat(cache): Phase 3 - data cache wrappers + index unread integration
- 2:
- 3: - extend _utils/cache-manager.js: getOrFetch now supports SWR (swrMs)
- 4: - add api-cache/unread.js and integrate into pages/index/index.vue (cached unread count)
- 5: - add api-cache/profile.js (getMyProfile) and api-cache/tags.js (getAllTags) - usable by pages later
- 6: - kept behavior backward compatible; no cloudfunctions changed
- 7:

===== FILE: .pending_commits/2025-10-11-commit-04.txt =====

- 1: feat(cache): user profile page uses unified CacheManager (event-driven invalidation)
- 2:
- 3: - add api-cache/user-profile.js: getUserInfo/getUserPosts + invalidate helpers (no TTL, manual invalidation)
- 4: - integrate pages/user-profile/user-profile.vue to load from cache and listen to global events:
- 5: - avatar-updated → invalidate user info and refresh
- 6: - post-created → invalidate user posts and refresh
- 7: - favorite-changed → invalidate user posts and refresh
- 8: - add api-cache/events.md documenting event names and expected payloads
- 9:
- 10: Behavior: personal homepage no longer re-fetches on每次进入; 仅头像更换/发帖/收藏变更触发刷新。
- 11: No cloudfunctions changed.
- 12:

===== FILE: .pending_commits/2025-10-11-commit-05.txt =====

- 1: feat(cache): integrate unified cache into My Profile page (posts + favorites) with event-driven invalidation
- 2:
- 3: - add api-cache/my.js: getMyInfo/getMyPosts/getMyFavorites + invalidate helpers (no TTL, manual invalidation)
- 4: - pages/profile/profile.vue:
- 5: - import cache helpers and switch loadMyPosts/loadFavorites to cached versions (early-return usage)
- 6: - bind uni events in checkLoginAndFetchData():
- 7: * avatar-updated → invalidateMyInfo + refresh info

```
8:      * post-created → invalidateMyPosts + refresh posts
9:      * favorite-changed → invalidateMyFavorites + refresh favorites (only for My Profile)
10: - no cloudfunctions changed; behavior respects your rule:
11:   * user-profile: refresh on avatar/post only
12:   * My Profile: also refresh on favorite changes
13:
```

===== FILE: .pending_commits/2025-10-11-commit-06.txt =====

```
1: feat(events): emit favorite-changed on collect success; emit avatar-updated on profile save
2:
3: - pages/post-detail/post-detail.vue: in onFavoriteSuccess, emit
4:   uni.$emit('favorite-changed', { userId, postId, favored: true })
5: - pages/profile-edit/profile-edit.vue: after successful save, emit
6:   uni.$emit('avatar-updated', { userId })
7: - pages/add/add.vue: after publish success, emit
8:   uni.$emit('post-created', { userId }) [already added in prior step]
9:
10: Note: user-profile listens to avatar/post only; My Profile listens to avatar/post/favorite.
11:
```

===== FILE: .pending_commits/2025-10-11-commit-07.txt =====

```
1: feat(events): centralize emits + hook folder-selector
2:
3: - add utils/events.js with EVENTS and emit helpers
4: - pages/add/add.vue: use emitPostCreated on publish success
5: - pages/profile-edit/profile-edit.vue: use emitAvatarUpdated on save success
6: - pages/profile/profile.vue: use emitFavoriteChanged on remove favorite
7: - components/folder-selector/folder-selector.vue: on confirmFavorite success, emit favorite-changed globally
8:
9: Notes:
10: - user-profile listens only avatar-updated/post-created;
11: - My Profile listens avatar-updated/post-created/favorite-changed.
12:
```

===== FILE: .pending_commits/2025-10-11-commit-08.txt =====

```
1: feat(cache): integrate CacheManager for home/discover/tag and clean old code
2:
3: - Home feed: add api-cache/home-posts.js (TTL 90s + SWR 45s); index.vue uses getHomePosts in getPostList/getIndexData
4: - Discover: api-cache/discover.js wired; index.vue loadRecommendationPosts uses cached feed
5: - Tag page: api-cache/tag-posts.js + hydrate/warm integration in tag-filter.vue
6: - Cleanup: removed legacy direct call branch in index.vue; reduced dataCache usage (refresh/getIndexData); syncLikeStatus...
7: - Images: keep hydrate + warm for cloud:// URLs
8:
9: How to commit once Git is available:
10:   git add -A
11:   git commit -F .pending_commits/2025-10-11-commit-08.txt
```

===== FILE: ARCHITECTURE.md =====

```
1: ## 项目架构总览
2:
```

3: 本项目基于 uni-app 构建, 前端以 Vue2 为主 (包含 Vue3 兼容代码), 后端使用腾讯云 CloudBase/微信云开发的云函数体系。页面通...

4:

5: ### 技术栈

6: - **前端**: uni-app (Vue2 主体, 含 Vue3 分支初始化逻辑)、原生 tabBar

7: - **云端**: 腾讯云 CloudBase (TCB)、微信云开发 (`wx-server-sdk`)

8: - **依赖**: `@cloudbase/js-sdk`、`wx-server-sdk`

9:

10: ### 启动与全局状态

11: - 全局入口在 `main.js`:

12: - 初始化 TCB: `tcb.init({ env, auth: { persistence: 'local' } })`, 并挂载到 `Vue.prototype.\$tcb`

13: - 拦截 `uniCloud.callFunction` 与 `tcb.callFunction`, 在调用时自动注入 `openid`, 并在未登录时阻断除 `login` 以外的云函数

14: - `App.vue`: 应用启动时执行匿名认证→调用 `login`→读取/创建用户→写入 `globalData` 与本地缓存, 形成登录态 (`openid`、`us...

15:

16: ### 路由与导航

17: - `pages.json` 定义页面与 tabBar:

18: - 主要页签: `pages/index/index` (广场)、`pages/poem/poem` (路)、`pages/mountain/mountain` (山)、`pages/profile/profi...

19: - 其余功能页: 登录/注册、详情、搜索、收藏、消息、反馈、图片管理等

20:

21: ### 页面层 (示例: 广场页 `pages/index/index.vue`)

22: - 职责:

23: - 列表加载/分页 (首屏骨架屏、滚动预加载、底部加载提示)

24: - 点赞 (本地乐观更新 + 云端校正) 与消息角标

25: - 发现页切换 (左右滑动) 与图片预载/占位防抖

26: - 数据来源: 通过 `this.\$tcb.callFunction({ name: 'getPostList', data: { skip, limit } })` 获取帖子列表, 并在前端补全 `ima...

27:

28: ### 云函数层 (Node, 位于 `functions/`)

29: - 配置: `cloudbaserc.json` 列出全部函数、内存与超时配置, 统一部署到 `envId`

30: - 示例: `functions/getPostList/index.js`

31: - 鉴权: 从 `cloud.getWXContext()` 或入参中解析 `openid`, 缺失则返回 `NO\_OPENID`

32: - 查询: `posts` 聚合查询, 可选 `isPoem`、`isOriginal`、`tag` 筛选; 关联 `users` (作者信息)、`comments` (评论数)、`vot...

33: - 媒体: 批量将 `cloud://` fileID 通过 `cloud.getTempFileURL` 转为临时直链 (头像/帖子图片/诗歌背景)

34: - 返回: `{ success: true, posts }`, 字段包含 `authorName`、`authorAvatar`、`commentCount`、`isVoted` 等

35: - 其余典型函数: 登录/用户 (`login`、`getUserProfile`、`updateUserProfile`、`createUser`)、内容 (`getPostDetail`、`search...

36:

37: ### 数据与权限

38: - 主要集合: `users`、`posts`、`comments`、`votes\_log`、`messages` 等

39: - 身份: 默认匿名认证; 前端调用云函数时透传 `openid`; 后端以 `getWXContext()` 兜底

40: - 媒体: 云存储 fileID 统一在服务端转换临时 URL 后返回, 提高前端直链显示性能

41:

42: ### 运行与部署

43: - 本地: 使用 HBuilderX 打开根目录, 安装依赖后可运行到 H5/小程序/App

44: - 云端: 依据 `cloudbaserc.json` 部署函数; 数据库集合与权限请按对应函数说明创建和配置

45:

46: ### 关键路径速览

47: - 入口初始化: `main.js` → 挂载 `\$tcb`、拦截云函数调用, 透传 `openid`

48: - 登录流程: `App.vue` → 匿名认证 → `login` → 写入 `globalData`/缓存

49: - 列表加载: `pages/index/index.vue` → `getPostList` → 前端展示优化与缓存

50: - 点赞流程: 前端乐观更新 → 云函数 `vote` → 缓存/列表同步

51:

52: ### 点赞实现与一致性

53: ##### 实现概览

54: - 前端: `pages/index/index.vue` 的 `onVote`

55: - 防重入: 使用 `votingInProgress[postId]` 阻挡重复点击

56: - 乐观更新: 即时切换 `isVoted`、`votes`, 并计算 `likeIcon`

57: - 失败回滚: 云函数失败/网络异常时回滚到原状态

58: - 成功后: 以服务端返回 `votes`/`isLiked` 校正, 并写入 `dataCache`

59: - 服务端: `functions/vote/index.js`

60: - 通过 `votes\_log` 是否存在来判定点赞/取消, 分别对 `posts.votes` 做 `\_.inc(±1)`

61: - 点赞时写入一条 `messages` 通知 (自赞不通知)

62: - 最终读取 `posts` 最新 `votes` 返回 `{ success, votes, isLiked }`

63:

64: ##### 一致性现状

65: - 列表、搜索、推荐与详情云函数统一以 `votes\_log` 计算 `isVoted`, 以 `posts.votes` 返回票数, 口径一致。

66: - 前端设置了本地节流与失败回滚, 降低单端多击带来的 UI 偏差。

67:

68: ##### 潜在问题 (并发/幂等)

69: - 跨端/多请求并发: 同一用户对同一帖子并发触发点赞/取消, 当前服务端逻辑采用“先查后改”的两步操作, 缺少事务与唯一性约束, ...

70: - 并发双“点赞”各自未读到对方插入的 `votes\_log`, 导致重复插入与累计 `votes`。

71: - 并发“点赞/取消”交错, 产生脏写, `votes` 与 `votes\_log` 不一致。

72: - 唯一键缺失: `votes\_log` 未声明复合唯一索引 (`\_openid, postId, type`), 难以依赖数据库层防重复。

73:

74: ##### 改进建议 (服务端为主)

75: 1) 增加复合唯一索引 (推荐)

76: - 对 `votes\_log(\_openid, postId, type)` 建立唯一索引, 数据库层去重。

77: 2) 使用事务包裹“查/增删/计数更新” (强一致)

78: - `const tx = await db.startTransaction();` 在事务内:

79: - 读取 `votes\_log` 是否存在;

80: - 依据存在与否执行 `add/remove`;

81: - 同时对 `posts.votes` 执行 `\_.inc(±1)`;

82: - 提交事务, 异常回滚重试。

83: 3) 基于幂等键的“插入若不存在/删除若存在” (次优, 弱一致)

84: - 尝试以“组合ID”方式写入 `votes\_log` (如 `\${\_openid}\_\${postId}\_post`) 避免重复;

85: - 更新 `posts.votes` 前重新读一次 `votes\_log` 确认最终态。

86: 4) 客户端补充防抖/限速

87: - 已有 `votingInProgress`; 可在多端登录场景下增加最小操作间隔 (如 500ms)。

88:

89: ##### 验收要点

90: - 并发压测下: `votes` 与 `votes\_log` 一致; `isVoted` 与实际日志一致; 重复请求不导致票数偏移。

91: - 列表与详情刷新后口径一致; 切页返回后点赞状态不闪烁。

92:

93: ##### 用户标识统一规范 (迁移后)

94: - 唯一主标识: 只使用 `openid` (含系统字段 `\_openid`)。

95: - 前端持久化: `globalData.openid` 与本地 `userOpenId`; `userInfo.\_openid = openid` 一并缓存。

96: - 登录覆盖: 匿名认证仅用于启动期; 登录成功后用真实 `openid` 覆盖全局与缓存。

97: - 云端判定: 所有 `isVoted`/权限判断仅以 `\_openid`。

98:

99: ##### 点赞状态保持策略 (前端)

100: - 等待登录完成: 首屏 `onLoad` 调用 `waitForLoginThenInit()`, 待 `globalData.\_loginProcessCompleted && openid` 后再拉取帖...

101: - 统一判定口径: 服务端 `getPostList/getPostDetail/...` 均基于 `votes\_log` 按 `\_openid+postId+type` 计算 `isVoted` (已统...

102: - 登录持久化: 登录成功时将 `openid` 与 `userInfo.\_openid` 写入本地并恢复全局, 登录页支持自动跳转 (已实现)。

103: - 乐观更新与校正: 前端先本地切换, 再以云函数返回的 `votes/isLiked` 校正并同步缓存 (已存在)。
104:
105:

===== FILE: DOCUMENTATION.md =====

```
1: # 回车键项目文档索引
2:
3: ## □ 文档结构说明
4:
5: 本文档提供了回车键项目的完整文档索引, 帮助开发者快速找到所需的技术文档和指南。
6:
7: ---
8:
9: ## □ 核心文档 (必读)
10:
11: ### [README.md] (. /README.md)
12: - **内容**: 项目完整介绍, 包括功能特性、技术栈、快速开始指南
13: - **适用**: 新项目成员、项目评估、快速上手
14: - **更新频率**: 随版本更新
15:
16: ### [ARCHITECTURE.md] (. /ARCHITECTURE.md)
17: - **内容**: 系统架构设计, 技术选型, 模块划分
18: - **适用**: 架构理解、技术决策、模块开发
19: - **更新频率**: 架构变更时更新
20:
21: ### [项目开发总结.md] (. /项目开发总结.md)
22: - **内容**: 开发过程经验总结, 问题解决方案, 最佳实践
23: - **适用**: 问题排查、开发参考、经验借鉴
24: - **更新频率**: 重大问题解决后更新
25:
26: ---
27:
28: ## □ 性能优化文档
29:
30: ### [PERFORMANCE_OPTIMIZATION.md] (. /PERFORMANCE_OPTIMIZATION.md)
31: - **内容**: 综合性能优化指南, 包含前端、后端、缓存系统优化
32: - **适用**: 性能优化实施、系统调优、用户体验提升
33: - **更新频率**: 持续更新
34:
35: ### [统一缓存与一致性改造计划.md] (. /统一缓存与一致性改造计划.md)
36: - **内容**: 详细的缓存系统设计方案和实施计划
37: - **适用**: 缓存系统理解、扩展维护
38: - **更新频率**: 缓存策略调整时更新
39:
40: ---
41:
42: ## □ 技术文档 (docs目录)
43:
44: ### □ 系统设计文档
45: - [docs/REFACTOR_OPTIMIZATION_PLAN.md] (. /docs/REFACTOR_OPTIMIZATION_PLAN.md) - 重构优化计划
```

46: - [docs/REFACTOR_REUSE_PLAN.md] (. /docs/REFACTOR_REUSE_PLAN.md) - 代码复用重构计划
47:
48: ### □□ 云开发生文档
49: - [docs/cloud-functions-deployment-status.md] (. /docs/cloud-functions-deployment-status.md) - 云函数部署状态
50: - [docs/backfill-cloud-function-guide.md] (. /docs/backfill-cloud-function-guide.md) - 数据回填云函数指南
51: - [docs/database-schema-improved.md] (. /docs/database-schema-improved.md) - 改进的数据库设计
52:
53: ### □ 认证与授权文档
54: - [docs/GITHUB_AUTH_FLOW.md] (. /docs/GITHUB_AUTH_FLOW.md) - GitHub认证流程
55: - [docs/GITHUB_OAUTH_SETUP.md] (. /docs/GITHUB_OAUTH_SETUP.md) - GitHub OAuth设置
56: - [docs/H5_GITHUB_AUTH.md] (. /docs/H5_GITHUB_AUTH.md) - H5端GitHub认证
57: - [docs/URL_SCHEME_CONFIG.md] (. /docs/URL_SCHEME_CONFIG.md) - URL方案配置
58:
59: ### □ 缓存系统文档
60: - [docs/cache-mechanism-review.md] (. /docs/cache-mechanism-review.md) - 缓存机制审查
61: - [docs/cache-sharing-usage.md] (. /docs/cache-sharing-usage.md) - 缓存共享使用指南
62: - [docs/cross-page-cache-sharing.md] (. /docs/cross-page-cache-sharing.md) - 跨页面缓存共享
63:
64: ### □ 日志与监控文档
65: - [docs/view-log-analysis.md] (. /docs/view-log-analysis.md) - 浏览日志分析
66: - [docs/view-log-implementation-plan.md] (. /docs/view-log-implementation-plan.md) - 日志系统实施计划
67: - [docs/view-log-single-record-design.md] (. /docs/view-log-single-record-design.md) - 单条日志记录设计
68:
69: ---
70:
71: ## □□ 开发工具文档
72:
73: ### 云函数文档
74: - [functions/README.md] (. /functions/README.md) - 云函数开发指南
75: - [functions/AuthHandler/README.md] (. /functions/AuthHandler/README.md) - 认证处理器文档
76:
77: ### API缓存文档
78: - [api-cache/events.md] (. /api-cache/events.md) - 事件系统文档
79:
80: ---
81:
82: ## □ 文档使用指南
83:
84: ### □ 快速查找
85: - **新手入门**: README.md → ARCHITECTURE.md → 项目开发总结.md
86: - **性能优化**: PERFORMANCE_OPTIMIZATION.md → 相关缓存文档
87: - **功能开发**: 对应模块的架构文档 + 相关技术文档
88: - **问题排查**: 项目开发总结.md → 相关问题文档
89:
90: ### □ 文档维护
91: - **新增功能**: 同步更新README.md和ARCHITECTURE.md
92: - **性能优化**: 更新PERFORMANCE_OPTIMIZATION.md和相关缓存文档
93: - **问题解决**: 更新项目开发总结.md
94: - **架构变更**: 更新ARCHITECTURE.md和相关设计文档
95:

```
96: ### □ 文档关联
97: ```
98: 核心文档
99: |—— README.md (项目总览)
100: |—— ARCHITECTURE.md (架构设计)
101: |—— 项目开发总结.md (实施经验)
102: |—— PERFORMANCE_OPTIMIZATION.md (性能指南)
103:
104: 技术文档 (docs/)
105: |—— 系统设计文档
106: |—— 云开发文档
107: |—— 认证授权文档
108: |—— 缓存系统文档
109: |—— 日志监控文档
110:
111: 实现文档
112: |—— functions/ (云函数)
113: |—— api-cache/ (缓存实现)
114: |—— utils/ (工具函数)
115: ```
116:
117: ---
118:
119: ## □ 文档规范
120:
121: ### □ 文档标准
122: - 使用清晰的结构和标题层级
123: - 提供实用的代码示例
124: - 包含必要的使用说明和注意事项
125: - 保持文档与代码同步更新
126:
127: ### □ 编写指南
128: - 使用Markdown格式
129: - 代码块标明语言类型
130: - 重要概念使用加粗强调
131: - 复杂操作提供步骤说明
132:
133: ### □ 更新机制
134: - 代码变更时同步更新相关文档
135: - 定期审查文档的准确性和完整性
136: - 重大功能更新时修订核心文档
137:
138: ---
139:
140: ## □ 问题反馈
141:
142: 如果在使用过程中遇到文档相关问题，可以通过以下方式反馈：
143:
144: - 查看相关文档的最新版本
145: - 在项目仓库中提交Issue
```

146: - 联系项目维护团队
147:
148: ---
149:
150: *最后更新: 2025-11-16*
151: *维护者: 回车键开发团队*

===== FILE: PERFORMANCE_OPTIMIZATION.md =====

```
1: # 回车键项目性能优化指南
2:
3: ## □ 概述
4:
5: 本文档整合了回车键项目的性能优化策略、缓存系统设计和用户体验改进建议，为项目的持续优化提供全面指导。
6:
7: ---
8:
9: ## □ 前端性能优化
10:
11: ### 1. 图片加载优化
12:
13: #### 1.1 智能图片懒加载
14: ```javascript
15: // utils/imageLazyLoader.js
16: class ImageLazyLoader {
17:   constructor() {
18:     this.observer = null;
19:     this.loadedImages = new Set();
20:     this.initIntersectionObserver();
21:   }
22:
23:   initIntersectionObserver() {
24:     if (typeof IntersectionObserver !== 'undefined') {
25:       this.observer = new IntersectionObserver((entries) => {
26:         entries.forEach(entry => {
27:           if (entry.isIntersecting) {
28:             this.loadImage(entry.target);
29:           }
30:         });
31:       }, {
32:         rootMargin: '50px 0px', // 提前50px开始加载
33:         threshold: 0.1
34:       });
35:     }
36:   }
37:
38:   observeImage(imgElement, src) {
39:     if (this.observer) {
40:       imgElement.dataset.src = src;
41:       this.observer.observe(imgElement);
42:     } else {
```

```

43:         imgElement.src = src;
44:     }
45: }
46: }
47: ```
48:
49: ##### 1.2 优化图片预加载策略
50: ```javascript
51: // 改进 imageOptimizer.js
52: class ImageOptimizer {
53:     constructor() {
54:         this.cache = new Map();
55:         this.preloadQueue = [];
56:         this.maxConcurrent = 3;
57:         this.currentLoading = 0;
58:     }
59:
60:     async preloadImages(urls, priority = 'normal') {
61:         const urlsToPreload = urls.slice(0, 10);
62:
63:         for (const url of urlsToPreload) {
64:             if (this.cache.has(url)) continue;
65:
66:             while (this.currentLoading >= this.maxConcurrent) {
67:                 await new Promise(resolve => setTimeout(resolve, 100));
68:             }
69:
70:             this.currentLoading++;
71:             this.loadImageWithRetry(url, 3).finally(() => {
72:                 this.currentLoading--;
73:             });
74:         }
75:     }
76: }
77: ```
78:
79: ### 2. 数据缓存优化
80:
81: ##### 2.1 多级缓存策略
82: ```javascript
83: // AdvancedDataCache.js
84: class AdvancedDataCache {
85:     constructor() {
86:         this.memoryCache = new Map();
87:         this.storageCache = new Map();
88:         this.cacheStats = new Map();
89:         this.maxMemorySize = 50;
90:     }
91:
92:     set(key, data, options = {}) {

```

```
93:     const { expiry = 300000, priority = 'normal', persist = true } = options;
94:
95:     const cacheItem = {
96:         data,
97:         timestamp: Date.now(),
98:         expiry,
99:         priority,
100:         accessCount: 0,
101:         lastAccess: Date.now()
102:     };
103:
104:     this.memoryCache.set(key, cacheItem);
105:     this.cacheStats.set(key, { hits: 0, misses: 0 });
106:
107:     if (persist) {
108:         try {
109:             uni.setStorageSync(key, cacheItem);
110:         } catch (e) {
111:             console.warn('持久化缓存失败:', e);
112:         }
113:     }
114:
115:     this.cleanup();
116: }
117:
118: get(key) {
119:     let item = this.memoryCache.get(key);
120:
121:     if (!item) {
122:         try {
123:             item = uni.getStorageSync(key);
124:             if (item) {
125:                 this.memoryCache.set(key, item);
126:             }
127:         } catch (e) {
128:             console.warn('读取持久化缓存失败:', e);
129:         }
130:     }
131:
132:     if (!item) {
133:         this.recordMiss(key);
134:         return null;
135:     }
136:
137:     if (Date.now() - item.timestamp > item.expiry) {
138:         this.remove(key);
139:         this.recordMiss(key);
140:         return null;
141:     }
142:
```

```

143:         item.accessCount++;
144:         item.lastAccess = Date.now();
145:         this.recordHit(key);
146:
147:         return item.data;
148:     }
149: }
150: ```
151:
152: ### 3. 页面渲染优化
153:
154: #### 3.1 组件复用和状态保持
155: ```javascript
156: export default {
157:     data() {
158:         return {
159:             componentPool: new Map(),
160:             preRenderCount: 3
161:         };
162:     },
163:
164:     methods: {
165:         switchToPost(index) {
166:             const postComponent = this.getOrCreatePostComponent(index);
167:             this.updatePostContent(postComponent, this.postList[index]);
168:         },
169:
170:         getOrCreatePostComponent(index) {
171:             if (this.componentPool.has(index)) {
172:                 return this.componentPool.get(index);
173:             }
174:
175:             const component = this.createPostComponent();
176:             this.componentPool.set(index, component);
177:             return component;
178:         },
179:
180:         preRenderNext() {
181:             const nextIndex = this.currentPostIndex + 1;
182:             if (nextIndex < this.postList.length) {
183:                 this.preloadPostContent(nextIndex);
184:             }
185:         }
186:     }
187: };
188: ```
189:
190: ---
191:
192: ## □ 统一缓存系统设计

```

```
193:
194: ### 1. CacheManager架构
195:
196: #### 1.1 核心API设计
197: ```typescript
198: type CacheOptions = { ttlMs?: number; swrMs?: number };
199: type Namespace = {
200:   get: (key: string) => Promise<any> | any;
201:   set: (key: string, val: any, opts?: CacheOptions) => Promise<void> | void;
202:   delete: (key: string) => Promise<void> | void;
203:   clear: () => Promise<void> | void;
204:   getOrFetch: <T>(key: string, loader: () => Promise<T>, opts?: CacheOptions) => Promise<T>;
205: };
206: ```
207:
208: #### 1.2 命名空间管理
209: ```javascript
210: // 主要缓存命名空间
211: const cacheNamespaces = {
212:   // 列表/分页缓存 (TTL 90s + SWR 45s)
213:   'posts:home': '广场首页分页',
214:   'posts:discover': '发现页推荐',
215:   'posts:tag:<tag>': '标签筛选分页',
216:
217:   // 资料/详情缓存
218:   'me:info': '我的资料 (TTL 30min + SWR 5min) ',
219:   'me:posts': '我的帖子 (仅事件失效) ',
220:   'me:favorites': '我的收藏 (仅事件失效) ',
221:   'profiles:user': '他人资料 (仅事件失效) ',
222:   'userPosts:<userId>': '他人主页帖子 (仅事件失效) ',
223:
224:   // 基础数据缓存
225:   'tags': '标签列表 (TTL 30min + SWR 5min) ',
226:   'unread': '未读数 (TTL 60s + SWR 60s) ',
227:
228:   // 媒体缓存
229:   'fileUrls': 'cloud:// → https 临时URL映射 (TTL 55min) ',
230:   'avatars': '用户头像/昵称/个签 (TTL 24h) ',
231:   'follow:<currentUserId>': '关注状态 (TTL 1h) '
232: };
233: ```
234:
235: ### 2. 事件驱动的缓存一致性
236:
237: #### 2.1 全局事件系统
238: ```javascript
239: // utils/events.js
240: const EVENTS = {
241:   POST_CREATED: 'post-created',
242:   AVATAR_UPDATED: 'avatar-updated',
```

```
243:     FAVORITE_CHANGED: 'favorite-changed'
244: };
245:
246: // 事件触发器
247: export function emitPostCreated(postData) {
248:     uni.$emit(EVENTS.POST_CREATED, postData);
249: }
250:
251: export function emitAvatarUpdated(userId, avatarData) {
252:     uni.$emit(EVENTS.AVATAR_UPDATED, { userId, avatarData });
253: }
254:
255: export function emitFavoriteChanged(data) {
256:     uni.$emit(EVENTS.FAVORITE_CHANGED, data);
257: }
258: ```
259:
260: ##### 2.2 缓存失效策略
261: ```javascript
262: // api-cache/events.js
263: export function setupCacheEventBridges() {
264:     // 发帖事件
265:     uni.$on('post-created', () => {
266:         cacheManager.namespace('posts:home').clear();
267:         cacheManager.namespace('posts:discover').clear();
268:     });
269:
270:     // 头像更新事件
271:     uni.$on('avatar-updated', ({ userId }) => {
272:         cacheManager.namespace('profiles:user').clear();
273:         cacheManager.namespace(`userPosts:${userId}`).clear();
274:     });
275:
276:     // 收藏变更事件
277:     uni.$on('favorite-changed', () => {
278:         // 根据需要实现相关缓存失效
279:     });
280: }
281: ```
282:
283: ### 3. SWR (Stale-While-Revalidate) 策略
284:
285: ##### 3.1 SWR实现
286: ```javascript
287: class CacheManager {
288:     async getOrFetch(key, loader, { ttlMs, swrMs } = {}) {
289:         const cached = this.get(key);
290:
291:         if (cached) {
292:             // 如果还在SWR窗口内，异步刷新
```

```

293:         if (this.isStale(key) && swrMs) {
294:             this.backgroundRefresh(key, loader);
295:         }
296:         return cached;
297:     }
298:
299:     // 缓存未命中, 直接加载
300:     return this.loadAndCache(key, loader, { ttlMs });
301: }
302:
303: async backgroundRefresh(key, loader) {
304:     try {
305:         const fresh = await loader();
306:         this.set(key, fresh, { ttlMs: this.getOriginalTtl(key) });
307:     } catch (error) {
308:         console.warn('后台刷新失败:', error);
309:     }
310: }
311: }
312: ```
313:
314: ---
315:
316: ## □ 后端性能优化
317:
318: ### 1. 云函数优化
319:
320: #### 1.1 数据预聚合
321: ```javascript
322: // 使用聚合管道优化查询
323: const result = await db.collection('posts').aggregate()
324:   .match({ isOriginal: true })
325:   .addFields({
326:     hotScore: {
327:       $add: [
328:         { $multiply: ['$votes', 2] },
329:         { $multiply: ['$commentCount', 5] }
330:       ]
331:     },
332:     timeWeight: {
333:       $divide: [
334:         { $subtract: [new Date(), '$createTime'] },
335:         86400000
336:       ]
337:     }
338:   })
339:   .sort({ hotScore: -1, createTime: -1 })
340:   .lookup({
341:     from: 'users',
342:     localField: '_openid',

```

```
343:         foreignField: '_openid',
344:         as: 'author'
345:     })
346:     .end();
347: ```
348:
349: ##### 1.2 批量临时URL获取
350: ```javascript
351: async function batchGetTempUrls(fileIds) {
352:     const batchSize = 20;
353:     const batches = [];
354:
355:     for (let i = 0; i < fileIds.length; i += batchSize) {
356:         batches.push(fileIds.slice(i, i + batchSize));
357:     }
358:
359:     const results = await Promise.all(
360:         batches.map(batch =>
361:             cloud.getTempFileURL({ fileList: batch })
362:         )
363:     );
364:
365:     return results.flatMap(result => result.fileList);
366: }
367: ```
368:
369: ### 2. 数据库优化
370:
371: ##### 2.1 索引配置
372: ```javascript
373: const indexes = {
374:     posts: [
375:         { isOriginal: 1, createTime: -1 },
376:         { _openid: 1, createTime: -1 },
377:         { votes: -1, createTime: -1 },
378:         { tags: 1, createTime: -1 }
379:     ],
380:     users: [
381:         { _openid: 1 },
382:         { poemId: 1 }
383:     ],
384:     comments: [
385:         { postId: 1, createTime: -1 },
386:         { _openid: 1, createTime: -1 }
387:     ]
388: };
389: ```
390:
391: —
392:
```

```
393: ## □ 用户体验优化
394:
395: ### 1. 交互体验优化
396:
397: #### 1.1 页面切换动画
398: ```css
399: .page-transition {
400:   transition: transform 0.3s cubic-bezier(0.4, 0, 0.2, 1);
401: }
402:
403: .page-enter-active {
404:   transform: translateX(0);
405: }
406:
407: .page-leave-active {
408:   transform: translateX(-100%);
409: }
410: ```
411:
412: #### 1.2 手势操作优化
413: ```javascript
414: export default {
415:   data() {
416:     return {
417:       touchStartX: 0,
418:       touchStartY: 0,
419:       touchStartTime: 0,
420:       swipeThreshold: 50,
421:       swipeTimeThreshold: 300
422:     };
423:   },
424:
425:   methods: {
426:     touchStart(e) {
427:       this.touchStartX = e.touches[0].clientX;
428:       this.touchStartY = e.touches[0].clientY;
429:       this.touchStartTime = Date.now();
430:     },
431:
432:     touchEnd(e) {
433:       const deltaX = e.changedTouches[0].clientX - this.touchStartX;
434:       const deltaY = e.changedTouches[0].clientY - this.touchStartY;
435:       const deltaTime = Date.now() - this.touchStartTime;
436:
437:       if (Math.abs(deltaX) > Math.abs(deltaY) &&
438:         Math.abs(deltaX) > this.swipeThreshold &&
439:         deltaTime < this.swipeTimeThreshold) {
440:
441:         if (deltaX > 0) {
442:           this.swipeRight();
```

```

443:         } else {
444:             this.swipeLeft();
445:         }
446:     }
447: }
448: }
449: };
450: ```
451:
452: ### 2. 加载状态优化
453:
454: #### 2.1 智能加载管理
455: ```javascript
456: class LoadingManager {
457:     constructor() {
458:         this.loadingStates = new Map();
459:         this.timeouts = new Map();
460:     }
461:
462:     showLoading(key, message = '加载中...', timeout = 10000) {
463:         if (this.loadingStates.has(key)) return;
464:
465:         this.loadingStates.set(key, true);
466:
467:         const timeoutId = setTimeout(() => {
468:             uni.showLoading({
469:                 title: message,
470:                 mask: true
471:             });
472:         }, 300);
473:
474:         this.timeouts.set(key, timeoutId);
475:
476:         setTimeout(() => {
477:             this.hideLoading(key);
478:         }, timeout);
479:     }
480:
481:     hideLoading(key) {
482:         if (!this.loadingStates.has(key)) return;
483:
484:         const timeoutId = this.timeouts.get(key);
485:         if (timeoutId) {
486:             clearTimeout(timeoutId);
487:             this.timeouts.delete(key);
488:         }
489:
490:         uni.hideLoading();
491:         this.loadingStates.delete(key);
492:     }

```

```
493: }
494: ```
495:
496: ---
497:
498: ## □ 性能监控
499:
500: ### 1. 性能指标收集
501: ```javascript
502: class PerformanceMonitor {
503:   constructor() {
504:     this.metrics = new Map();
505:   }
506:
507:   startTiming(key) {
508:     this.metrics.set(key, {
509:       startTime: Date.now(),
510:       endTime: null,
511:       duration: null
512:     });
513:   }
514:
515:   endTiming(key) {
516:     const metric = this.metrics.get(key);
517:     if (metric) {
518:       metric.endTime = Date.now();
519:       metric.duration = metric.endTime - metric.startTime;
520:     }
521:   }
522:
523:   reportMetrics() {
524:     const report = Array.from(this.metrics.entries()).map(([key, metric]) => ({
525:       key,
526:       duration: metric.duration,
527:       timestamp: Date.now()
528:     }));
529:
530:     this.sendReport(report);
531:   }
532: }
533: ```
534:
535: ### 2. 错误监控
536: ```javascript
537: class ErrorMonitor {
538:   constructor() {
539:     this.errors = [];
540:     this.maxErrors = 100;
541:   }
542:
```

```
543:     captureError(error, context = {}) {
544:         const errorInfo = {
545:             message: error.message,
546:             stack: error.stack,
547:             context,
548:             timestamp: Date.now(),
549:             userAgent: navigator.userAgent,
550:             url: window.location.href
551:         };
552:
553:         this.errors.push(errorInfo);
554:
555:         if (this.errors.length > this.maxErrors) {
556:             this.errors.shift();
557:         }
558:
559:         this.sendError(errorInfo);
560:     }
561: }
562: ...
563:
564: ---
565:
566: ## ☐ 实施优先级
567:
568: ### 高优先级（立即实施）
569: 1. **统一缓存系统** - 已实现，持续优化
570: 2. **图片懒加载** - 显著减少初始加载时间
571: 3. **SWR策略** - 提升用户体验
572: 4. **事件驱动失效** - 保证数据一致性
573:
574: ### 中优先级（近期实施）
575: 1. **云函数查询优化** - 减少服务器响应时间
576: 2. **批量操作优化** - 提升数据处理效率
577: 3. **性能监控实现** - 持续优化基础
578: 4. **错误处理完善** - 提升应用稳定性
579:
580: ### 低优先级（长期规划）
581: 1. **高级缓存策略** - 进一步优化性能
582: 2. **智能预加载** - 预测用户行为
583: 3. **个性化推荐优化** - 提升内容质量
584: 4. **多平台深度优化** - 平台特定优化
585:
586: ---
587:
588: ## ☐ 预期效果
589:
590: 实施以上优化措施后，预期可以获得：
591:
592: ### 性能提升
```

```
593: - **页面加载时间减少 40-60%**
594: - **图片加载速度提升 50%**
595: - **云函数响应时间减少 30%**
596: - **内存使用优化 25%**
597:
598: ### 用户体验提升
599: - **页面切换更流畅**
600: - **加载状态更友好**
601: - **数据一致性更好**
602: - **交互响应更及时**
603:
604: ---
605:
606: *本文档整合了项目中的性能优化策略和最佳实践，为项目的持续优化提供指导。*
```

```
===== FILE: README.md =====
1: # 回车键 - 诗歌创作与分享平台
2:
3: <div align="center">
4:
5: ![回车键](https://img.shields.io/badge/回车键-诗歌平台-blue)
6: ![uni-app](https://img.shields.io/badge/uni--app-Vue2-green)
7: ![CloudBase](https://img.shields.io/badge/CloudBase-腾讯云-orange)
8: ![多平台](https://img.shields.io/badge/多平台-小程序%20H5%20App-purple)
9:
10: 一个优雅的诗歌创作、分享与交流平台，支持多端运行
11:
12: [功能特性](#-功能特性) □ [技术栈](#-技术栈) □ [快速开始](#-快速开始) □ [项目结构](#-项目结构) □ [部署指南](#-部署指南)
13:
14: </div>
15:
16: ## □ 项目简介
17:
18: **回车键**是一个专注于诗歌创作与分享的社交平台，为用户提供优雅的创作环境和丰富的互动功能。项目采用uni-app框架开发，支持…
19:
20: ### □ 核心特色
21:
22: - **诗界面**：简洁优雅的UI设计，营造沉浸式创作氛围
23: - **多端同步**：一次开发，多端运行，数据实时同步
24: - **智能推荐**：基于用户喜好的个性化内容推荐
25: - **作品集管理**：支持创建个人作品集，分类管理创作内容
26: - **社区互动**：点赞、评论、关注等完整的社交功能
27:
28: ## □ 功能特性
29:
30: ### □ 创作功能
31: - **诗歌创作**：支持富文本编辑，自定义背景色和文字颜色
32: - **拼贴诗**：创新的拼贴诗歌创作模式
33: - **草稿保存**：自动保存草稿，支持离线编辑
34: - **图片上传**：支持多图片上传，自动压缩优化
```

```
35:
36: ### □ 社交互动
37: - **广场浏览**: 发现页展示精选内容
38: - **个性化推荐**: 基于用户行为的智能推荐
39: - **点赞评论**: 完整的互动体系
40: - **关注系统**: 关注感兴趣的作者
41: - **消息通知**: 实时消息推送
42:
43: ### □ 内容管理
44: - **作品集**: 创建个人作品集, 分类管理
45: - **收藏夹**: 收藏喜欢的作品
46: - **标签系统**: 为作品添加标签, 便于分类
47: - **搜索功能**: 全文搜索, 快速找到内容
48:
49: ### □ 用户系统
50: - **多端登录**: 支持微信登录和账号密码登录
51: - **个人资料**: 自定义头像、昵称、签名
52: - **隐私设置**: 支持匿名发布
53: - **数据统计**: 查看个人创作和互动数据
54:
55: ## □ 技术栈
56:
57: ### 前端技术
58: - **框架**: uni-app (Vue 2)
59: - **UI组件**: 自定义组件 + 原生组件
60: - **状态管理**: Vuex + 本地存储
61: - **样式**: SCSS + 响应式设计
62:
63: ### 后端技术
64: - **云服务**: 腾讯云开发 (CloudBase)
65: - **云函数**: Node.js + wx-server-sdk
66: - **数据库**: 云数据库 (MongoDB)
67: - **存储**: 云存储 (文件上传)
68:
69: ### 开发工具
70: - **IDE**: HBuilderX
71: - **版本控制**: Git
72: - **部署**: CloudBase CLI
73:
74: ## □ 快速开始
75:
76: ### 环境要求
77: - Node.js >= 14.0.0
78: - HBuilderX >= 3.0.0
79: - 微信开发者工具 (小程序开发)
80:
81: ### 安装步骤
82:
83: 1. **克隆项目**
84: ```bash
```

```
85: git clone https://github.com/your-username/回车键_uni.git
86: cd 回车键_uni
87: ```
88:
89: 2. **安装依赖**
90: ```bash
91: npm install
92: ```
93:
94: 3. **配置云开发环境**
95:   - 在腾讯云开发控制台创建环境
96:   - 修改 `cloudbaserc.json` 中的环境ID
97:   - 配置数据库权限
98:
99: 4. **部署云函数**
100: ```bash
101: # 安装 CloudBase CLI
102: npm install -g @cloudbase/cli
103:
104: # 登录腾讯云
105: cloudbase login
106:
107: # 部署云函数
108: cloudbase functions:deploy
109: ```
110:
111: 5. **运行项目**
112:   - 使用 HBuilderX 打开项目
113:   - 选择运行平台 (H5/小程序/App)
114:   - 点击运行
115:
116: ### 开发模式
117:
118: ```bash
119: # H5开发
120: npm run dev:h5
121:
122: # 小程序开发
123: npm run dev:mp-weixin
124:
125: # App开发
126: npm run dev:app-plus
127: ```
128:
129: ## □ 项目结构
130:
131: ```
132: 回车键_uni/
133: |—— pages/          # 页面文件
134: |   |—— index/      # 广场首页
```

```

135: | | | | poem-square/      # 诗歌广场
136: | | | | mountain/      # 山（发现页）
137: | | | | profile/       # 个人中心
138: | | | | post-detail/   # 帖子详情
139: | | | | add/           # 发布页面
140: | | | | ...
141: | | | | components/     # 组件库
142: | | | | | custom-tabbar/ # 自定义标签栏
143: | | | | | folder-selector/ # 文件夹选择器
144: | | | | | portfolio-selector/ # 作品集选择器
145: | | | | | ...
146: | | | | functions/     # 云函数
147: | | | | | login/       # 登录
148: | | | | | getPostList/ # 获取帖子列表
149: | | | | | vote/        # 点赞
150: | | | | | addComment/  # 添加评论
151: | | | | | ...
152: | | | | | utils/       # 工具函数
153: | | | | | | likeService.js # 点赞服务
154: | | | | | | cloudCall.js  # 云函数调用
155: | | | | | | ...
156: | | | | static/        # 静态资源
157: | | | | api-cache/     # API缓存
158: | | | | App.vue        # 应用入口
159: | | | | main.js         # 主文件
160: | | | | pages.json      # 页面配置
161: | | | | manifest.json   # 应用配置
162: | | | | | cloudbaserc.json # 云开发配置
163: | | | | ```
164:
165: ## ☐ 配置说明
166:
167: ### 云开发配置
168:
169: 在 `cloudbaserc.json` 中配置你的云开发环境：
170:
171: ```json
172: {
173:   "envId": "your-env-id",
174:   "framework": {
175:     "name": "uni-app",
176:     "plugins": {
177:       "uni-app": {
178:         "version": "2.0.0"
179:       }
180:     }
181:   }
182: }
183: ```
184:

```

```
185: ### 数据库集合
186:
187: 项目使用以下主要集合:
188: - `users` - 用户信息
189: - `posts` - 帖子内容
190: - `comments` - 评论数据
191: - `votes_log` - 点赞记录
192: - `messages` - 消息通知
193: - `portfolios` - 作品集
194: - `favorites` - 收藏记录
195:
196: ## □ 多端适配
197:
198: ### 微信小程序
199: - 支持微信登录
200: - 使用微信云开发
201: - 适配小程序UI规范
202:
203: ### H5端
204: - 使用CloudBase SDK
205: - 支持浏览器访问
206: - 响应式设计
207:
208: ### App端
209: - 支持原生功能
210: - 相机拍照
211: - 文件上传
212:
213: ## □ 部署指南
214:
215: ### 云函数部署
216:
217: ```bash
218: # 部署所有云函数
219: cloudbase functions:deploy
220:
221: # 部署单个云函数
222: cloudbase functions:deploy login
223:
224: # 查看部署状态
225: cloudbase functions:list
226: ```
227:
228: ### 前端部署
229:
230: #### H5部署
231: ```bash
232: # 构建H5版本
233: npm run build:h5
234:
```

235: # 部署到服务器
236: # 将 dist/build/h5 目录上传到服务器
237: ```
238:
239: ##### 小程序发布
240: 1. 使用微信开发者工具打开项目
241: 2. 点击"上传"按钮
242: 3. 在微信公众平台提交审核
243:
244: ##### App发布
245: 1. 使用HBuilderX云打包
246: 2. 生成安装包
247: 3. 上传到应用商店
248:
249: ## ☐ 贡献指南
250:
251: 我们欢迎所有形式的贡献!
252:
253: ### 如何贡献
254:
255: 1. Fork 本仓库
256: 2. 创建你的特性分支 (`git checkout -b feature/AmazingFeature`)
257: 3. 提交你的修改 (`git commit -m 'Add some AmazingFeature'`)
258: 4. 推送到分支 (`git push origin feature/AmazingFeature`)
259: 5. 打开一个 Pull Request
260:
261: ### 开发规范
262:
263: - 遵循 Vue.js 官方风格指南
264: - 使用 ESLint 进行代码检查
265: - 编写清晰的提交信息
266: - 添加必要的注释和文档
267:
268: ## ☐ 许可证
269:
270: 本项目采用 MIT 许可证 - 查看 LICENSE 文件了解详情
271:
272: ## ☐ 致谢
273:
274: 感谢以下开源项目和服务:
275: - [uni-app](https://uniapp.dcloud.io/) - 跨平台开发框架
276: - [腾讯云开发](https://cloud.tencent.com/product/tcb) - 云服务支持
277: - [Vue.js](https://vuejs.org/) - 前端框架
278:
279: ## ☐ 联系我们
280:
281: - 项目地址: [GitHub](https://github.com/your-username/回车键_uni)
282: - 问题反馈: [Issues](https://github.com/your-username/回车键_uni/issues)
283: - 邮箱: your-email@example.com
284:

```
285: ---
286:
287: <div align="center">
288:
289: **让诗歌在指尖流淌，让创意在云端绽放** □
290:
291: Made with □□ by 回车键团队
292:
293: </div>
```

===== FILE: README_FONT_CLOUD.md =====

```
1: # 字体云存储动态加载方案
2:
3: ## 概述
4: 将字体文件从本地静态资源迁移到腾讯云存储，实现按需动态下载，显著减小应用包大小。
5:
6: ## 技术架构
7:
8: ### 1. 文件结构
9: ```
10: |—— utils/
11: |   |—— fontManager.js          # 字体管理核心
12: |   |—— fontUploader.js        # 字体上传工具（开发用）
13: |—— components/
14: |   |—— FontSelectorModal.vue   # 字体选择器（已更新支持下载）
15: |—— pages/
16: |   |—— font-manager/          # 字体管理页面
17: |       |—— font-manager.vue
18: |—— static/fonts/              # 仅保留默认字体
19: |   |—— Huiwen-mincho.otf      # 汇文明朝（默认字体）
20: ```
21:
22: ### 2. 核心功能
23:
24: #### FontManager 类
25: - **字体缓存管理**：LRU策略，50MB上限
26: - **动态下载**：从腾讯云存储按需下载
27: - **本地存储**：缓存到 `wx.env.USER_DATA_PATH/fonts/`
28: - **版本控制**：支持字体更新和校验
29:
30: #### 字体配置
31: ```javascript
32: const FONT_CONFIG = {
33:   'Huiwen-mincho': { displayName: '汇文明朝', isDefault: true },
34:   '文楷': { displayName: '文楷', size: 18456789 },
35:   '蒲瓜正楷体': { displayName: '蒲瓜正楷体', size: 15234567 },
36:   // ... 其他字体
37: };
38: ```
39:
```

```
40: ## 部署步骤
41:
42: ### 1. 配置腾讯云存储
43:
44: #### 1.1 创建云存储桶
45: ```bash
46: # 在腾讯云开发控制台创建存储桶
47: # 或使用 CLI 工具
48: tcb storage:create-bucket fonts-bucket
49: ```
50:
51: #### 1.2 上传字体文件
52: ```bash
53: # 方法一：使用控制台上传
54: # 1. 登录腾讯云开发控制台
55: # 2. 进入云存储
56: # 3. 创建 fonts/ 目录
57: # 4. 上传字体文件
58:
59: # 方法二：使用 CLI 工具
60: tcb storage:upload ./static/fonts/文楷.ttf fonts/文楷.ttf
61: tcb storage:upload ./static/fonts/蒲公英正楷体.ttf fonts/蒲公英正楷体.ttf
62: tcb storage:upload ./static/fonts/龙藏体.ttf fonts/龙藏体.ttf
63: tcb storage:upload ./static/fonts/小小皓体.ttf fonts/小小皓体.ttf
64: tcb storage:upload ./static/fonts/南西雅致黑.ttf fonts/南西雅致黑.ttf
65: tcb storage:upload ./static/fonts/字体圈欣意吉祥宋.ttf fonts/字体圈欣意吉祥宋.ttf
66: tcb storage:upload ./static/fonts/汇文明朝-蒲公英版.ttf fonts/汇文明朝-蒲公英版.ttf
67: ```
68:
69: #### 1.3 配置 CDN 加速（推荐）
70: ```javascript
71: // 在 fontManager.js 中更新 URL
72: const CLOUD_FONT_BASE_URL = 'https://your-cdn-domain.com/fonts';
73: // 或使用直接存储URL
74: const CLOUD_FONT_BASE_URL = 'https://your-env-id.tcb.qcloud.la/fonts';
75: ```
76:
77: ### 2. 更新应用配置
78:
79: #### 2.1 修改字体管理器配置
80: ```javascript
81: // utils/fontManager.js
82: const CLOUD_FONT_BASE_URL = 'https://your-actual-storage-url.com/fonts';
83:
84: // 更新字体文件大小（获取实际文件大小）
85: const FONT_CONFIG = {
86:   '文楷': { size: 18456789 }, // 替换为实际大小
87:   // ... 其他字体配置
88: };
89: ```
```

```
90:
91: ##### 2.2 清理本地字体文件
92: ```bash
93: # 删除非默认字体文件, 保留 Huiwen-mincho.otf
94: rm static/fonts/文楷.ttf
95: rm static/fonts/蒲瓜正楷体.ttf
96: rm static/fonts/龙藏体.ttf
97: rm static/fonts/小小皓体.ttf
98: rm static/fonts/南西雅致黑.ttf
99: rm static/fonts/字体圈欣意吉祥宋.ttf
100: rm static/fonts/汇文明朝-蒲瓜版.ttf
101: ```
102:
103: ### 3. 功能验证
104:
105: ##### 3.1 测试下载功能
106: 1. 删除应用数据重新安装
107: 2. 进入分享页面选择非默认字体
108: 3. 观察下载进度和缓存效果
109: 4. 测试网络异常时的降级处理
110:
111: ##### 3.2 缓存管理测试
112: 1. 访问字体管理页面: `/pages/font-manager/font-manager`
113: 2. 测试下载、删除、清空功能
114: 3. 验证缓存大小限制和LRU清理
115:
116: ## 技术优势
117:
118: ### 包体积优化
119: - **前**: 8个字体文件  $\approx$  80MB
120: - **后**: 1个默认字体  $\approx$  2.5MB
121: - **减少**: 约77.5MB (97%减少)
122:
123: ### 用户体验
124: - **首次使用**: 立即可用默认字体
125: - **字体切换**: 进度条显示下载状态
126: - **离线使用**: 已下载字体无需网络
127: - **智能缓存**: 自动清理最少使用字体
128:
129: ### 技术特性
130: - **渐进加载**: 先显示默认字体, 后台下载
131: - **错误处理**: 下载失败自动降级
132: - **版本控制**: 支持字体文件更新
133: - **跨平台**: 支持小程序、APP、H5
134:
135: ## 监控和维护
136:
137: ### 1. 性能监控
138: ```javascript
139: // 添加字体下载监控
```

```
140: fontManager.on('downloadStart', (fontFamily) => {
141:     // 上报下载开始事件
142: });
143:
144: fontManager.on('downloadSuccess', (fontFamily, duration) => {
145:     // 上报下载成功和耗时
146: });
147:
148: fontManager.on('downloadError', (fontFamily, error) => {
149:     // 上报下载错误
150: });
151: ```
152:
153: ### 2. 缓存统计
154: ```javascript
155: // 获取缓存使用情况
156: const stats = fontManager.getCacheStats();
157: console.log('缓存统计:', stats);
158: ```
159:
160: ### 3. 用户行为分析
161: - 字体使用频率统计
162: - 下载成功率监控
163: - 缓存命中率分析
164:
165: ## 注意事项
166:
167: ### 1. 网络环境
168: - 确保云存储 CDN 覆盖用户地区
169: - 设置合理的超时时间（建议30秒）
170: - 提供网络异常提示
171:
172: ### 2. 存储权限
173: - 配置云存储读取权限
174: - 确保匿名用户可访问字体文件
175: - 定期检查存储配额使用情况
176:
177: ### 3. 兼容性
178: - 测试不同平台的字体加载
179: - 验证低端设备的性能表现
180: - 确保网络受限环境的可用性
181:
182: ## 成本分析
183:
184: ### 存储成本
185: - 字体文件总大小: ~80MB
186: - 腾讯云存储费用: ~0.1元/月
187:
188: ### 流量成本
189: - 单次字体下载: 2-20MB
```

190: - 月活用户下载: 根据实际使用情况
191: - CDN 加速费用: 按流量计费
192:
193: ### ROI 收益
194: - 应用包大小减少97%
195: - 应用商店审核通过率提升
196: - 用户下载转化率提升
197: - 更新版本流量成本降低
198:
199: ## 后续优化
200:
201: ### 1. 智能预加载
202: ```javascript
203: // 根据用户历史使用偏好预加载字体
204: fontManager.preloadUserFavorites();
205: ```
206:
207: ### 2. 增量更新
208: ```javascript
209: // 支持字体文件增量更新
210: fontManager.checkForUpdates();
211: ```
212:
213: ### 3. 压缩优化
214: ```javascript
215: // 字体文件压缩和子集化
216: fontManager.loadFontSubset(fontFamily, characters);
217: ```
218:
219: ---
220:
221: **实施时间**: 预计2-3天
222: **维护成本**: 低
223: **技术风险**: 低
224: **预期收益**: 显著减小包体积, 提升用户体验

===== FILE: api-cache/events.md =====

1: 约定的全局事件 (仅文档, 便于接入各页面触发缓存失效)
2:
3: - avatar-updated
4: - 参数: { userId }
5: - 处理: invalidateUserInfo(userId); fileUrlCache.invalidate(oldFileId)
6:
7: - post-created
8: - 参数: { userId }
9: - 处理: invalidateUserPosts(userId); 可选刷新首页第一页缓存
10:
11: - favorite-changed
12: - 参数: { userId, postId, favored }
13: - 处理: invalidateUserPosts(userId) (或仅失效含该 post 的页键)

- 2: - 鸿蒙平台 新增 打开 url 错误时输出
- 3: - iOS平台 修复 语法报黄问题
- 4: ## 1.1.2 (2025-03-20)
- 5: - 更新 支持鸿蒙
- 6: ## 1.1.1 (2024-12-16)
- 7: - 修复 canOpenURL 在安卓端可能会报类型错误的问题
- 8: ## 1.1.0 (2024-12-06)
- 9: - 新增 canOpenURL UTS API, 可用此API判断url是否可以跳转
- 10: ## 1.0.1 (2024-11-13)
- 11: - 修复 Android 打开部分 schema 时没有跳转到目标应用的 Bug
- 12: ## 1.0.0 (2024-04-25)
- 13: - 更新 在 Android 和 iOS 上打开链接的 UTS API

===== FILE: uni_modules/uts-openSchema/readme.md =====

```
1: # uts-openSchema
2:
3: 打开链接, 支持:
4:
5: 1. 打开外部 App
6: 2. 使用浏览器打开链接
7: 3. 打开地图到指定地点
8: 4. ...
9:
10: ## 使用
11:
12: 1. 安装此插件
13: 2. 在要使用的地方 `import` 导入
14: ```ts
15: import { openSchema, canOpenURL } from '@uni_modules/uts-openSchema'
16: ```
17: 3. 直接调用 `openSchema` 方法:
18: ```ts
19: // #ifdef UNI-APP-X
20: // 使用外部浏览器打开指定URL
21: openSchema('https://uniapp.dcloud.io/uni-app-x')
22:
23: // #ifdef APP-ANDROID
24: // Android 使用应用商店打开指定App
25: openSchema('market://details?id=com.tencent.mm')
26:
27: // Android 打开地图坐标
28: // 可以先用canOpenURL判断是否安装了地图软件
29: if (canOpenURL('androidamap://')) {
30:   openSchema('androidamap://viewMap?sourceApplication=Hello%20uni-app&poiname=DCloud&lat=39.9631018208&lon=116.34061352...')
31: } else {
32:   console.log('未安装高德地图')
33: }
34: // #endif -->
35:
36: // #ifdef APP-IOS
```

```
37: // 打开 AppStore 到搜索页
38: openSchema('itms-apps://search.itunes.apple.com/WebObjects/MZSearch.woa/wa/search?media=software&lterm=')
39:
40: // 打开 iOS 地图坐标
41: openSchema('http://maps.apple.com/?q=Mexican+Restaurant&sl=50.894967,4.341626&z=10&t=s')
42: // #endif -->
43:
44: // #endif -->
45: ```
46:
47: ### 参数
48:
49: - openSchema(url: string) // `url`: 要打开的链接 `必填` `不为空字符串`
50:
51: ## 相关开发文档
52:
53: [UTS 语法](https://uniapp.dcloud.net.cn/tutorial/syntax-uts.html)
54:
55: [UTS API 插件](https://uniapp.dcloud.net.cn/plugin/uts-plugin.html)
56:
57: [UTS 组件插件](https://uniapp.dcloud.net.cn/plugin/uts-component.html)
58:
59: [Hello UTS](https://gitcode.net/dcloud/hello-uts)
```

===== FILE: uni_modules/uts-progressNotification/changelog.md =====

```
1: ## 1.1.2 (2025-02-10)
2: 修复某些情况通过点击通知消息无法拉起App的bug
3: ## 1.1.1 (2024-09-03)
4: 去除TypeScript警告
5: ## 1.1.0 (2024-03-08)
6: 修复uniapp打包报错问题
7: ## 1.0.9 (2024-02-29)
8: 去除代码过时警告
9: ## 1.0.8 (2023-12-21)
10: 去除app-ios目录
11: ## 1.0.7 (2023-12-11)
12: 去除无用代码
13: ## 1.0.6 (2023-12-11)
14: 修改文档
15: ## 1.0.5 (2023-12-11)
16: 1. 修改插件名称
17: 2. 修改插件引入方式为import导入
18: ## 1.0.4 (2023-11-30)
19: 1. createNotificationProgress增加`onClick`回调
20: 2. 修复在小米部分系统上，通知消息会归类于不重要通知的bug
21: ## 1.0.3 (2023-11-28)
22: 更新截图
23: ## 1.0.2 (2023-11-28)
24: 修改资源的包名
25: ## 1.0.1 (2023-11-28)
```

26: 更新文档
27: ## 1.0.0 (2023-11-28)
28: Android通知栏显示进度插件

```
===== FILE: uni_modules/uts-progressNotification/readme.md =====
1: # uts-progressNotification
2:
3: ## 使用说明
4:
5: Android平台创建显示进度的通知栏消息
6:
7: **注意: 需要自定义基座, 否则点击通知栏消息不会拉起应用**
8:
9: ### 导入
10:
11: 需要import导入插件
12:
13: ### createNotificationProgress(options : CreateNotificationProgressOptions) : void,
14:
15: 创建显示进度的通知栏消息
16:
17: 参数说明
18:
19: ```
20: export type CreateNotificationProgressOptions = {
21:   /**
22:    * 通知标题
23:    * @defaultValue 应用名称
24:    */
25:   title?: string | null
26:   /**
27:    * 通知内容
28:    */
29:   content: string,
30:   /**
31:    * 进度
32:    */
33:   progress: number,
34:   /**
35:    * 点击通知消息回调
36:    * @defaultValue null
37:    */
38:   onClick?: (() => void) | null
39: }
40: ```
41:
42: ### finishNotificationProgress(options: FinishNotificationProgressOptions) : void
43:
44: 完成时调用的API, 比如下载完成后需要显示下载完成并隐藏进度时调用。
45:
```

```

46: 参数说明
47:
48:
49: ```
50: export type FinishNotificationProgressOptions = {
51:   /**
52:    * 通知标题
53:    * @defaultValue 应用名称
54:    */
55:   title?: string | null
56:   /**
57:    * 通知内容
58:    */
59:   content: string,
60:   /**
61:    * 点击通知消息回调
62:    */
63:   onClick: () => void
64: }
65: ```
66:
67:
68: ### cancelNotificationProgress() : void
69:
70: 取消通知消息显示
71:

```

===== FILE: uni_modules/zp-mixins/changelog.md =====

```

1: ## 1.0.6 (2023-04-12)
2: 为防冲突，给selectComponent两个函数添加前缀zp
3: ## 1.0.5 (2023-03-28)
4: 修复ts项目找不到setData的问题
5: ## 1.0.4 (2023-03-07)
6: 修复小程序报错
7: ## 1.0.3 (2023-02-26)
8: 修复pageLifetimes
9: ## 1.0.2 (2023-01-06)
10: 增加使用文档
11: ## 1.0.1 (2022-12-07)
12: 更新文档
13: ## 1.0.0 (2022-12-07)
14: init

```

===== FILE: uni_modules/zp-mixins/readme.md =====

```

1: # zp-mixins
2: 本插件，基本无法单独使用。
3:
4: 为方便miniprogram-to-uniapp(小程序转uniapp项目)工具后续升级，特单独摘出做成插件，便于后续更新。
5:
6:

```

```
7: 注意:
8:
9: 1. 本插件仅限于miniprogram-to-uniapp(小程序转uni-app项目)转换工具使用,
10:
11: 2. 如果是新项目或uni-app项目, 建议还是使用vue/uni-app它自身支持的方式实现, 效率更高些。
12:
13: 3. 目前仅支持vue2/vue3的options写法。至于setup, 因其天生弱化mixin, 也无this, 建议直接import导入按需使用吧~
14:
15: # 安装
16:
17: 点击右侧 “使用 HBuilderX 导入插件” 按钮, 并选择对应项目。
18:
19: # 使用
20:
21: 在main.js文件里面增加引用, 即可全局使用这些函数, 如下所示:
22:
23: ```
24:
25: import App from './App'
26:
27: // 添加zpMixins
28: import zpMixins from '@uni_modules/zp-mixins/index.js'
29:
30: // #ifndef VUE3
31: import Vue from 'vue'
32:
33: Vue.use(zpMixins)
34:
35: Vue.config.productionTip = false
36: App.mpType = 'app'
37: const app = new Vue({
38:   ...App
39: })
40: app.$mount()
41: // #endif
42:
43: // #ifdef VUE3
44: import { createSSRApp } from 'vue'
45: export function createApp() {
46:   const app = createSSRApp(App)
47:   app.mixin(zpMixins)
48:   return {
49:     app
50:   }
51: }
52: // #endif
53: ```
54:
55: 注意: vue3项目仅支持Options API方式(setup API写法有点不同, 不支持这种方式使用, 建议直接通过import引入使用)。
56:
```

```
57: # 功能介绍
58:
59: ## pageLifetimes (组件所在页面的生命周期)
60: 处理组件里的pageLifetimes
61:
62: ## clone (深拷贝)
63: 深拷贝的简单版本，未处理循环引用。
64:
65: ## handleDataset
66: 用于处理dataset
67:
68: 在自定义组件的事件里，是获取不到e.currentTarget.dataset的
69:
70: 因此收集data-参数，手动传进去
71:
72: ## html2Escape 普通字符转换成转义符
73: 用于替换wxParse为mp-html时使用
74:
75: ## escape2Html 转义符转换成普通字符
76: 暂未用上
77:
78: ## parseEventDynamicCode
79: 解析事件里的动态函数名，这种没有()的函数名，在uniapp不被执行
80:
81: 比如: <view bindtap="{{openId==undefined?'denglu':'hy_to'}}">立即</view>
82:
83: ## getTabBar
84: 实现小程序自定义组件的this.getTabBar().setData() 函数
85:
86: ## getRelationNodes
87: 获取组件间关系this.getRelationNodes()
88:
89: 注意:
90:
91: 1. 须与p-f-unicom配合使用!
92:
93: 2. 并不能与小程序的getRelationNodes相提并论，因为底层实现不一样
94:
95: ## selectComponent
96: ## selectAllComponents
97: 抹平各平台差异，实现类似于小程序的selectComponent和selectAllComponents函数。
98:
99: 使用方式与小程序一致。
100:
101: 注: 新项目建议还是使用ref这种方式，获取组件实例。
102:
103: ## setData
104: 1. 实现setData函数，让uni-app也能支持使用setData函数。
105: 2. 实现微信“简易双向绑定”
106:
```

107: 注意: 本函数仅为小程序转换uniapp项目所作的支撑, 如uni-app项目里, 最好还是使用this.xx这种方式。

===== FILE: 文本.txt =====

```
1: 1. 为什么到处都是冰凌
2: 2. 因为此地是妆台
3: 3. 玫瑰即花香
4: 4. 塞尚的苹果我全都消受不来
5: 5. 黄昏织满了红雀的翅膀
6: 6. 她在那独自成婚
7: 7. 我却不肯蓝
8: 8. 石榴爆裂
9: 9. 我敞开我的花衣
10: 10. 时代的郁金香雷雨
11: 11. 悲伤的童年的甜美
12: 12. 它甜过草莓, 甜过蜜
13: 13. 星星或许会相视大笑
14: 14. 膨胀的虱子
15: 15. 我的河流和你的河流
16: 16. 饭后的一场睡眠
17: 17. 模糊而黯然神伤的镜子
18: 18. 你如同忧郁这个字
19: 19. 湖底其实无底
20: 20. 眼下是春天
21: 21. 梦像鸟儿一样飞了
22: 22. 猫太咸了
23: 23. 我安然而坐
24: 24. 青苔说了另一半
25: 25. 橄榄树林一阵悲风
26: 26. 影子不会死
27: 27. 我不问皑皑白雪
28: 28. 胭脂用尽时
29: 29. 争渡, 争渡
30: 30. 昨日之日不可留
31: 31. 飘飘何所似
32: 32. 此闲行
33: 33. 我做云
34: 34. 恐美人之迟暮
35: 35. 太阳天天新天天旧
36: 36. 但岸陆表示怀疑
37: 37. 通过我
38: 38. 一毛税也不用交
39: 39. 所有的城市都是船
40: 40. 它荒草萋萋
41: 41. 你不仅是安妮塔
42: 42. 马蹄正在踢着石头
43: 43. 做坏事像下雨
44: 44. 这位死了的女士的眼睛对我诉说
45: 45. 吻向虚空的齿列
46: 46. 一场樱花人生
47: 47. 感叹号太漫不经心
```

48: 48. 祝太阳万岁
49: 49. 只要你要
50: 50. 你这只人类的小舟
51: 51. 如果你吻我
52: 52. 鸢尾花开遍窗前和心窗
53: 53. 此身仍是草芥
54: 54. 一朵百合的馨香将弥漫河畔
55: 55. 这时天将大雪降在巴黎
56: 56. 好等你醒来与我相识
57: 57. 三等车厢的夏天
58: 58. 修辞而已
59: 59. 无性别的卵
60: 60. 一个看不见的热吻
61: 61. 一切被称作大地的
62: 62. 在电梯里想着乡野
63: 63. 生人即离别
64: 64. 打开、关闭、打开
65: 65. 因云和树的倒影不是云和树而受苦
66: 66. 食盐戏法
67: 67. 必要的真空
68: 68. 我是一汪黑水
69: 69. 地球颠倒两极
70: 70. 请把世界装进一只牡蛎
71: 71. 菩萨难写
72: 72.
73:
74:
75:
76:
77:

===== FILE: 日志.md =====

1: [广告] 20:59:05.728 uni实人认证,低成本核验用户身份、提升用户信任度、规避法律风险,详情
2: 20:59:05.769 项目 回车键_uni 开始编译
3: 20:59:06.915 当前项目的uniCloud使用的默认服务空间spaceId为: mp-85244698-6ebe-485e-8f19-2d36212a6ad0
4: 20:59:07.025 4.76
5: 20:59:07.025 请注意运行模式下,因日志输出、sourcemap以及未压缩源码等原因,性能和包体积,均不及发行模式。
6: 20:59:07.150 正在编译中...
7: 20:59:25.931 uts插件[uts-progressNotification]文件未发生变化,跳过编译
8: 20:59:25.931 ☐提示: uts插件[uts-progressNotification]依赖的原生配置或三方SDK在运行至标准基座时不能生效,如需正常调用请...
9: 20:59:31.571 项目 回车键_uni 编译成功。
10: 20:59:31.983 手机端调试基座版本号为4.76,版本号相同,跳过基座更新
11: 20:59:32.218 正在建立手机连接...
12: 20:59:34.157 正在同步手机端程序文件...
13: 20:59:35.468 同步手机端程序文件完成
14: 20:59:36.712 正在启动HBuilder调试基座...
15: 20:59:37.841 应用【回车键_uni】已启动
16: 20:59:38.319 [warn]: getApp() failed. Learn more: <https://uniapp.dcloud.io/collocation/frame/window?id=getapp>.
17: 20:59:38.319 [warn]: getApp() failed. Learn more: <https://uniapp.dcloud.io/collocation/frame/window?id=getapp>.
18: 20:59:38.351 ☐ [App] onLaunch 启动参数:, at App.vue:25

```
19: 20:59:38.351 [热更新] 当前为 APP-PLUS 环境, 准备获取版本信息... at App.vue:36
20: 20:59:38.351 [热更新] App ID:, __UNI__E0A1A41 at App.vue:40
21: 20:59:38.351 [App] onShow 触发 at App.vue:96
22: 20:59:38.351 [URL Scheme] 启动参数:, at App.vue:101
23: 20:59:38.366 [PlatformDetector] 环境检测结果:, [Object] {"platform":"app","isH5":false,"isApp":true,"isMiniProgram":f...
24: 20:59:38.367 [PlatformDetector] 使用TCB云函数调用方式 at utils/platformDetector.js:172
25: 20:59:38.367 [cloudCall] 云函数调用方式: tcb, 函数名: getUnreadMessageCount at utils/cloudCall.js:109
26: 20:59:38.367 [cloudCall][unread] 调用云函数 "getUnreadMessageCount" (尝试 1/1), [Object] {"openid":"ojYBd1_A3uCbQ1LGcHx...
27: 20:59:38.367 [invokeCloudFunction] 调用方式: tcb at utils/cloudCall.js:42
28: 20:59:38.367 [invokeCloudFunction] TCB实例:, 可用 at utils/cloudCall.js:46
29: 20:59:38.445 [cloudbase/js-sdk][INVALID_OPERATION]:every cloudbase instance should has only one auth object at node_modu...
30: 20:59:38.490 [热更新] plus.runtime.getProperty 回调已执行 at App.vue:43
31: 20:59:38.490 [热更新] App 版本号 (versionCode):, 110 at App.vue:48
32: 20:59:38.491 [热更新] WGT 资源包版本号 (wgtVersion):, 1.1.1 at App.vue:49
33: 20:59:38.491 [热更新] 当前完整版本信息:, [Object] {"appId":"__UNI__E0A1A41","appVersion":"110","wgtVersion":"1.1.1"} ...
34: 20:59:38.491 [hotUpdate] ===== 开始完整更新检查流程 ===== at utils/hotUpdate.js:286
35: 20:59:38.491 [hotUpdate] 第一步: 检查整包更新... at utils/hotUpdate.js:291
36: 20:59:38.491 [热更新] check-update: 开始检查更新... at uni_modules/uni-upgrade-center-app/utils/check-update.ts:8
37: 20:59:38.505 [热更新] call-check-version: 使用固定 App ID:, __UNI__E0A1A41 at uni_modules/uni-upgrade-center-app/util...
38: 20:59:38.505 [热更新] call-check-version: systemInfo.appId:, __UNI__E0A1A41, (仅供参考) at uni_modules/uni-upgrade-c...
39: 20:59:38.505 [热更新] call-check-version: plus.runtime.appid:, HBuilder, (仅供参考) at uni_modules/uni-upgrade-cente...
40: 20:59:38.506 [热更新] call-check-version: 使用 plus.runtime.appid 获取版本信息:, HBuilder at uni_modules/uni-upgrade-...
41: 20:59:38.537 [cloudbase/js-sdk][INVALID_OPERATION]:every cloudbase instance should has only one auth object at node_modu...
42: 20:59:38.552 [post-list-cache] 请求数据 - key:, page:0:size:5:poem:true:orig:true, forceRefresh:, [Boolean] false ,...
43: 20:59:38.567 [post-list-cache] 请求数据 - key:, page:0:size:5:poem:true:orig:false, forceRefresh:, [Boolean] false ...
44: 20:59:38.582 [热更新] call-check-version: plus.runtime.getProperty 回调, widgetInfo:, [Object] {"appid":"HBuilder","v...
45: 20:59:38.583 [热更新] call-check-version: App 版本号 (versionCode):, [Number] 110 at uni_modules/uni-upgrade-center-...
46: 20:59:38.583 [热更新] call-check-version: WGT 版本号:, 1.1.1, (如果为空则使用默认值 0.0.0) at uni_modules/uni-upgrad...
47: 20:59:38.583 [热更新] call-check-version: 准备调用云函数检查更新, 参数:, [Object] {"appid":"__UNI__E0A1A41","appVersi...
48: 20:59:38.583 [热更新] call-check-version: 调用云函数参数详情:, {
49:   "action": "checkVersion",
50:   "appid": "__UNI__E0A1A41",
51:   "appVersion": "110",
52:   "wgtVersion": "1.1.1"
53: } at uni_modules/uni-upgrade-center-app/utils/call-check-version.ts:44
54: 20:59:38.859 通用图片预加载尝试完毕:, /static/images/like.png at pages/splash/splash.vue:699
55: 20:59:38.859 通用图片预加载尝试完毕:, /static/images/liked.png at pages/splash/splash.vue:699
56: 20:59:40.977 [cloudCall][unread] 云函数 "getUnreadMessageCount" 调用成功, [Object] {"result":{"success":true,"count":0},...
57: 20:59:40.992 [unread-badge] 初始化未读数:, [Number] 0 at cache/stores/unread-badge.js:32
58: 20:59:42.982 预加载任务完成 at pages/splash/splash.vue:388
59: 20:59:42.982 等待用户点击进入按钮 at pages/splash/splash.vue:398
60: 20:59:43.167 [登录流程] getApp().globalData 已更新:, [Object] {"_loginProcessStarted":true,"openid":"ojYBd1_A3uCbQ1LG...
61: 20:59:43.167 [登录流程] 缓存验证成功, 用户已登录 at App.vue:358
62: 20:59:43.351 [热更新] call-check-version: 云函数调用成功, 返回结果:, [Object] {"code":0,"message":"当前版本已经是最...
63: 20:59:43.351 [热更新] call-check-version: 返回结果详情:, {
64:   "code": 0,
65:   "message": "当前版本已经是最新的, 不需要更新"
66: } at uni_modules/uni-upgrade-center-app/utils/call-check-version.ts:50
67: 20:59:43.351 [热更新] 云函数返回 code=0, 表示当前已是最新版本 at uni_modules/uni-upgrade-center-app/utils/call-check-v...
68: 20:59:43.351 [热更新] 这可能是由于: at uni_modules/uni-upgrade-center-app/utils/call-check-version.ts:62
```

```
69: 20:59:43.351 1. 数据库中的 version 没有同时大于 appVersion 和 wgtVersion at uni_modules/uni-upgrade-center-app/utills/call...
70: 20:59:43.352 2. 对于 wgt 包, 云函数要求 version 必须同时大于 appVersion("110") 和 wgtVersion("1.1.1") at uni_modules/uni-...
71: 20:59:43.352 3. 如果数据库中的 version 是 "1.0.2", appVersion 是 "104", 比较会失败 ("1.0.2" < "104") at uni_modules/uni-...
72: 20:59:43.352 [热更新] check-update: 版本检查返回 code:, [Number] 0 at uni_modules/uni-upgrade-center-app/utills/check...
73: 20:59:43.352 [热更新] check-update: 当前已是最新版本, 无需更新 at uni_modules/uni-upgrade-center-app/utills/check-updat...
74: 20:59:43.352 [hotUpdate] 官方插件返回:, [Object] {"code":0,"message":"当前版本已经是最新的, 不需要更新"} at utills/ho...
75: 20:59:43.352 [hotUpdate] 无整包更新 (code=null), 继续检查热更新... at utills/hotUpdate.js:309
76: 20:59:43.352 [hotUpdate] 第二步: 检查热更新... at utills/hotUpdate.js:317
77: 20:59:43.352 [hotUpdate] ===== 开始检查 WGT 热更新 ===== at utills/hotUpdate.js:75
78: 20:59:43.352 [hotUpdate] ----- 获取版本信息 ----- at utills/hotUpdate.js:25
79: 20:59:43.367 [hotUpdate] 当前平台:, android at utills/hotUpdate.js:28
80: 20:59:43.367 [hotUpdate] plus.runtime.appid:, HBuilder at utills/hotUpdate.js:31
81: 20:59:43.367 [hotUpdate] widgetInfo 完整信息:, {
82:   "appid": "HBuilder",
83:   "version": "1.1.1",
84:   "name": "poementer",
85:   "versionCode": "110",
86:   "description": "",
87:   "author": "",
88:   "email": "",
89:   "features": [
90:     "camera",
91:     "file",
92:     "io",
93:     "oauth",
94:     "uninview"
95:   ]
96: } at utills/hotUpdate.js:39
97: 20:59:43.367 [hotUpdate] ===== 本地版本信息汇总 ===== at utills/hotUpdate.js:49
98: 20:59:43.367 [hotUpdate] wgtVersion (资源版本):, 1.1.1 at utills/hotUpdate.js:50
99: 20:59:43.367 [hotUpdate] versionCode (整数版本):, 110 at utills/hotUpdate.js:51
100: 20:59:43.367 [hotUpdate] platform:, android at utills/hotUpdate.js:52
101: 20:59:43.367 [hotUpdate] name:, poementer at utills/hotUpdate.js:53
102: 20:59:43.367 [hotUpdate] appid:, HBuilder at utills/hotUpdate.js:54
103: 20:59:43.367 [hotUpdate] ===== at utills/hotUpdate.js:55
104: 20:59:43.367 [hotUpdate] 准备调用云函数 checkWgtUpdate at utills/hotUpdate.js:82
105: 20:59:43.367 [hotUpdate] 请求参数:, {
106:   "appid": "__UNI__E0A1A41",
107:   "wgtVersion": "1.1.1",
108:   "platform": "android"
109: } at utills/hotUpdate.js:83
110: 20:59:44.374 [hotUpdate] 云函数返回结果:, {
111:   "code": 0,
112:   "message": "当前已是最新版本",
113:   "hasUpdate": false,
114:   "currentVersion": "1.1.1",
115:   "latestVersion": "1.1.1"
116: } at utills/hotUpdate.js:98
117: 20:59:44.374 [hotUpdate] 无需更新, 原因:, 当前已是最新版本 at utills/hotUpdate.js:106
118: 20:59:44.374 [hotUpdate] 热更新检查结果:, [Object] {"code":0,"message":"当前已是最新版本","hasUpdate":false,"currentV...
```

```
119: 20:59:44.374 ☐ [hotUpdate] 当前已是最新版本, 无需更新 at utils/hotUpdate.js:347
120: 20:59:44.374 ☐ [hotUpdate] ===== 更新检查流程结束 ===== at utils/hotUpdate.js:370
121: 20:59:44.374 ☐ [热更新] 检查完成:, [Object] {"type":"latest","code":0,"message":"当前已是最新版本","hasUpdate":false,"c...
122: 20:59:54.817 [云函数:uni-upgrade-center] 运行日志: [云函数-热更新] 查询参数: {
123:   appid: '__UNI__E0A1A41',
124:   platform: 'Android',
125:   stable_publish: true,
126:   appVersion: '110',
127:   wgtVersion: '1.1.1'
128: }
129: [云函数-热更新] 查询结果: {
130:   recordCount: 2,
131:   records: [
132:     {
133:       appid: '__UNI__E0A1A41',
134:       platform: [Array],
135:       version: '1.1.1',
136:       type: 'wgt',
137:       stable_publish: true
138:     },
139:     {
140:       appid: '__UNI__E0A1A41',
141:       platform: [Array],
142:       version: '104',
143:       type: 'native_app',
144:       stable_publish: true
145:     }
146:   ]
147: }
148: [云函数-热更新] 检查 native_app 包更新: { dbVersion: '104', clientVersion: '110', appUpdate: false }
149: [云函数-热更新] 检查 native_app 版本是否匹配: {
150:   hasAppPackage: true,
151:   dbAppVersion: '104',
152:   clientAppVersion: '110',
153:   appVersionMatch: false
154: }
155: [云函数-热更新] native_app 版本不匹配, 无法进行 wgt 更新
156: 20:59:55.774 [云函数:checkWgtUpdate] 运行日志: ☐ [checkWgtUpdate] 收到请求: { appid: '__UNI__E0A1A41', wgtVersion: '1.1.1'...
157: ☐ [checkWgtUpdate] 数据库查询结果: {
158:   affectedDocs: 1,
159:   data: [
160:     {
161:       _id: '692840d14b92476632d221b0',
162:       is_silently: false,
163:       is_mandatory: false,
164:       appid: '__UNI__E0A1A41',
165:       name: 'poementer',
166:       title: '',
167:       contents: '测试',
168:       platform: [Array],
```

```
169:     version: '1.1.1',
170:     url: 'https://mp-85244698-6ebe-485e-8f19-2d36212a6ad0.cdn.bspapp.com/__UNI__E0A1A41_1764245669714_0.wgt',
171:     stable_publish: true,
172:     type: 'wgt',
173:     min_uni_version: '1.1.0',
174:     uni_platform: 'android',
175:     create_env: 'upgrade-center',
176:     create_date: 1764245714491
177:   }
178: ]
179: }
180: □ [checkWgtUpdate] 找到最新版本: {
181:   _id: '692840d14b92476632d221b0',
182:   is_silently: false,
183:   is_mandatory: false,
184:   appid: '__UNI__E0A1A41',
185:   name: 'poementer',
186:   title: '',
187:   contents: '测试',
188:   platform: [ 'Android' ],
189:   version: '1.1.1',
190:   url: 'https://mp-85244698-6ebe-485e-8f19-2d36212a6ad0.cdn.bspapp.com/__UNI__E0A1A41_1764245669714_0.wgt',
191:   stable_publish: true,
192:   type: 'wgt',
193:   min_uni_version: '1.1.0',
194:   uni_platform: 'android',
195:   create_env: 'upgrade-center',
196:   create_date: 1764245714491
197: }
198: □ [checkWgtUpdate] 版本检查: 客户端版本 1.1.1 >= min_uni_version 1.1.0
199: □ [checkWgtUpdate] 版本对比: 云端 1.1.1 === 客户端 1.1.1
200: 21:00:06.988 【poem-square】系统信息:, [Object] {"statusBarHeight":41,"safeAreaInsets":{"top":41,"right":0,"bottom":0,"l...
201: 21:00:06.988 【poem-square】使用状态栏高度作为安全区域:, [Number] 41 at pages/poem-square/poem-square.vue:252
202: 21:00:06.999 【poem-square】开始获取数据, callback:, undefined at pages/poem-square/poem-square.vue:275
203: 21:00:07.005 □ [poem-square-cache] 开始读取缓存, key:, page:0:size:10:poem:true:orig:true:exclAnon:true at pages/poem-s...
204: 21:00:07.005 □ [poem-square-cache] 检查持久化存储 at pages/poem-square/poem-square.vue:288
205: 21:00:07.020 □ [poem-square-cache] 所有缓存键:, [Object] ["page:0:size:5:poem:true:orig:true","page:0:size:5:poem:true:...
206: 21:00:07.020 □ [poem-square-cache] 匹配到的键:, page:0:size:10:poem:true:orig:true:exclAnon:true at pages/poem-square/p...
207: 21:00:07.034 □ [poem-square-cache] 从持久化读取的记录:, [Object] {"hasValue":true,"isArray":true,"length":10,"expireAt"...
208: 21:00:07.034 □ [poem-square-cache] 从持久化存储读取到缓存数据, 数量:, [Number] 10 at pages/poem-square/poem-square.vue...
209: 21:00:07.034 □ [poem-square-cache] 已恢复到内存缓存 at pages/poem-square/poem-square.vue:304
210: 21:00:07.034 □ [poem-square-cache] 缓存读取结果:, [Object] {"found":true,"source":"persistent","count":10} at pages/po...
211: 21:00:07.034 □ [poem-square-cache] 发现缓存数据, 立即显示, 数量:, [Number] 10 at pages/poem-square/poem-square.vue:330
212: 21:00:07.034 □ [poem-square-cache] 过滤匿名后的数量:, [Number] 10 at pages/poem-square/poem-square.vue:333
213: 21:00:07.034 □ [poem-square-cache] 已显示缓存数据, 准备后台刷新 at pages/poem-square/poem-square.vue:353
214: 21:00:07.345 □ [poem-square-cache] 开始后台刷新, 重置页码为0 at pages/poem-square/poem-square.vue:357
215: 21:00:07.345 □□□ 【poem-square】getPostList 开始, isLoadingMore:, [Boolean] false, isLoading:, [Boolean] false, ...
216: 21:00:07.345 □□□ 【poem-square】当前页码:, [Number] 0, PAGE_SIZE:, [Number] 10 at pages/poem-square/poem-square.v...
217: 21:00:07.345 □ [post-list-cache] 请求数据 - key:, page:0:size:10:poem:true:orig:true:exclAnon:true, forceRefresh:, [B...
218: 21:00:07.345 □□□ 【poem-square】获取到帖子数量:, [Number] 10 at pages/poem-square/poem-square.vue:461
```

219: 21:00:07.345 ☐☐☐ 【poem-square】当前页码:, [Number] 0, 现有列表长度:, [Number] 10 at pages/poem-square/poem-squar...
220: 21:00:07.345 ☐☐☐ 【poem-square】所有帖子的详细信息 (用于验证随机性): at pages/poem-square/poem-square.vue:465
221: 21:00:07.345 [1] ID: e18d05796928023c054e2f5f0e450da3, 创建时间: 2025-11-27T07:48:12.204Z, 标题: 金黄大道 at pages/poem-s...
222: 21:00:07.345 [2] ID: cd349d8f6927a622053a31e456367981, 创建时间: 2025-11-27T01:15:13.968Z, 标题: 寻欢 at pages/poem-squar...
223: 21:00:07.345 [3] ID: 8e2c0a2f6927a49f053475ff06dc148c, 创建时间: 2025-11-27T01:08:47.157Z, 标题: 路痴 at pages/poem-squar...
224: 21:00:07.345 [4] ID: def9fa84692759500528d6bc7a244a41, 创建时间: 2025-11-26T19:47:27.977Z, 标题: 句符 at pages/poem-squar...
225: 21:00:07.346 [5] ID: cd349d8f69270b65052b31956906a2a6, 创建时间: 2025-11-26T14:15:01.864Z, 标题: 回廊 at pages/poem-squar...
226: 21:00:07.346 [6] ID: 7f4b671369268d9e05192d497b4005fa, 创建时间: 2025-11-26T05:18:21.956Z, 标题: 即兴 at pages/poem-squar...
227: 21:00:07.346 [7] ID: def9fa8469268d290513feab0e816692, 创建时间: 2025-11-26T05:16:25.359Z, 标题: 即兴 at pages/poem-squar...
228: 21:00:07.346 [8] ID: da8988ea69268d10051e444e5cbbc494, 创建时间: 2025-11-26T05:16:00.021Z, 标题: 即兴 at pages/poem-squar...
229: 21:00:07.346 [9] ID: d84694f269268cdc051b987b3ca85234, 创建时间: 2025-11-26T05:15:08.777Z, 标题: 即兴 at pages/poem-squar...
230: 21:00:07.346 [10] ID: def9fa8469265f78050e960775e48565, 创建时间: 2025-11-26T02:01:28.814Z, 标题: 喜欢与不喜欢 at pages/p...
231: 21:00:07.346 ☐☐☐ 【poem-square】前3个帖子的时间:, [Object] [{"id":"e18d05796928023c054e2f5f0e450da3","createTime":"20...
232: 21:00:07.346 ☐☐☐ 【poem-square】后4个帖子 (应该是随机的) 的创建时间:, [Object] [{"id":"def9fa8469268d290513feab0e8166...
233: 21:00:07.346 ☐☐☐ 【poem-square】过滤匿名后的数量:, [Number] 10 at pages/poem-square/poem-square.vue:484
234: 21:00:07.346 ☐☐☐ 【poem-square】第一页刷新, 现有列表:, [Number] 10, 新数据:, [Number] 10 at pages/poem-square/poe...
235: 21:00:07.346 ☐☐☐ 【poem-square】真正的新帖子数量:, [Number] 0 at pages/poem-square/poem-square.vue:504
236: 21:00:07.346 ☐☐☐ 【poem-square】没有新帖子, 更新现有帖子数据 at pages/poem-square/poem-square.vue:512
237: 21:00:07.346 ☐☐☐ 【poem-square】最终列表长度:, [Number] 10, 页码:, [Number] 1 at pages/poem-square/poem-square.vu...
238: 21:00:07.346 【poem-square】数据处理完成 at pages/poem-square/poem-square.vue:541
239: 21:00:07.346 【poem-square】执行回调函数 at pages/poem-square/poem-square.vue:549
240: 21:00:07.346 ☐ [poem-square-cache] 后台刷新完成, 新数据:, [Number] 10 at pages/poem-square/poem-square.vue:360

==== FILE: 统一缓存与一致性改造计划.md =====

- 1: # 统一缓存与一致性改造计划 (uni-app × CloudBase)
- 2:
- 3: 本文给出“临时 URL 缓存 + 统一缓存工具 (CacheManager) + 跨端一致性”的实施路线, 按优先级分阶段落地, 确保简单易用、可灰度...
- 4:
- 5: — 版本: v1 (2025-10-11)
- 6:
- 7: ## 目标与原则
- 8:
- 9: - 目标:
- 10: - 提升首屏与滚动性能, 降低云函数与网络调用量。
- 11: - 保持多端一致性 (多设备/多平台), 允许短暂可控陈旧, 关键数据“近实时”。
- 12: - 对前端开发者暴露简洁稳定的 API, 便于后续功能扩展/重构。
- 13: - 原则:
- 14: - 客户端优先, 本地缓存先返回, 必要时后台刷新 (SWR)。
- 15: - 可观测: 有命中率/耗时统计与日志开关。
- 16: - 安全: 尊重服务端过期 (maxAge), 为临时 URL 预留刷新裕量。
- 17: - 可灰度、可回滚: 功能开关与回退预案完善。
- 18:
- 19: ## 优先级说明
- 20:
- 21: - P0: 强相关、立竿见影, 优先完成。
- 22: - P1: 结构化优化, 提升长期可维护性与一致性。
- 23: - P2: 增强与体验加分项。
- 24:
- 25: ## 路线图 (分阶段)
- 26:

27: ### Phase 1 (P0, 1-2 天): 临时 URL 缓存落地与接入
28:
29: - 任务
30: - 接入 `\_utils/file-url-cache.js` (已完成初版):
31: - 批处理 + TTL + 去重 + 多端兜底 (tcb / wx.cloud / 云函数)。
32: - 默认 TTL: 55 分钟; 安全提前量: ≥ 5 分钟 (从 1 分钟提升)。
33: - 页面集成:
34: - 在列表与详情页引入 `hydrateTempUrls(list)` , 批量映射 cloud:// $\rightarrow$ http(s)。
35: - 触底/预判下一页调用 `fileUrlCache.warm(nextIds)` 进行预热。
36: - 本机失效:
37: - 上传/更换头像/删除图片成功后, `fileUrlCache.invalidate(oldFileId)`。
38: - 版本化 URL (cache-busting):
39: - 展示层拼接 `?v=<updatedAt|version>` (头像、帖子图片), 避免跨设备“长 TTL 显旧图”。
40: - 交付物
41: - 工具: `\_utils/file-url-cache.js` (已存在)。
42: - 接入示例/混入: `\_utils/hydrate-temp-urls.js` (新增, 便于在各页直接复用)。
43: - 文档: 使用说明与最佳实践 (本计划 + 代码注释)。
44: - 验收标准
45: - 首屏与翻页图片加载云函数调用量下降 $\geq 30\%$ 。
46: - 页面未出现因 URL 过期导致的图片 404 (稳态)。
47: - 常用路径不增加白屏时长 (监控 P75/P95)。
48:
49: ### Phase 2 (P1, 2-4 天): 统一缓存工具 CacheManager
50:
51: - 任务
52: - 新增 `\_utils/cache-manager.js`:
53: - 多层缓存: 内存层 (Map/LRU) + 可选持久层 (uniStorage)。
54: - 命名空间: 按 `openid` 与业务划分 (如 `fileUrls:`, `profiles:`, `tags:`)。
55: - 统一 API:
56: - `namespace(name)` $\rightarrow$ 命名空间句柄
57: - `get(key) / set(key, val, { ttlMs }) / delete(key) / clear()`
58: - `getOrFetch(key, loader, { ttlMs, swrMs })` (SWR 支持)
59: - 容量与淘汰: 内存 LRU (条目上限), 持久层容量与清理策略。
60: - 观测: `hits/misses/evictions` 计数, `debug` 日志开关。
61: - 将 `file-url-cache` 的 Map 替换为 `cacheManager.namespace('fileUrls')` 存取 (对外 API 不变)。
62: - 提供通用工具:
63: - `stableKey(params)` (稳定 JSON 序列化)
64: - `withSWR()` 辅助函数
65: - 开关与配置:
66: - `CACHE\_DEBUG`、`CACHE\_MAX\_ITEMS`、`CACHE\_STORAGE\_ENABLED` 等配置位 (从 `manifest.json` 或环境注入)。
67: - 交付物
68: - 工具: `\_utils/cache-manager.js`、改造后的 `\_utils/file-url-cache.js`。
69: - 文档: API 说明与示例。
70: - 验收标准
71: - 替换后所有页面行为一致; `file-url-cache` 命中率提升、内存可控。
72: - 开启持久层后, 冷启动列表请求减少, 性能数据无明显回退。
73:
74: ### Phase 3 (P1, 3-5 天): 数据缓存与一致性增强
75:
76: - 任务

77: - 基于 CacheManager 实现以下业务缓存:
78: - 用户资料 (`profiles`): TTL 30 - 60 分钟, SWR 5 分钟。
79: - 标签列表 (`tags`): TTL 30 - 60 分钟, SWR 5 分钟。
80: - 未读数 (`unread`): TTL 30 - 60 秒, 页面 onShow 轻刷新。
81: - Feed 列表 (`postList`/`discover`): 分页分键, SWR 开启, 前后台切换无闪烁。
82: - 变更驱动失效:
83: - 点赞/评论/删除后, 失效当前帖子详情与所在分页条目键。
84: - 登出/切账号: 清空与用户相关的命名空间。
85: - 监控与告警:
86: - 前端埋点 (命中率/耗时) + 云函数侧请求量对比。
87: - 交付物
88: - `api-cache/\*.js` 示例与封装。
89: - 页面改造 PR (index, post-detail, profile 等)。
90: - 验收标准
91: - 页面返回/切换明显更快; 接口总请求量下降 $\geq 20\%$ 。
92: - 未读等易变数据 “近实时”, 无明显延迟感。
93:
94: ### Phase 4 (P1→P2, 2 - 4 天): 后端配合与灰度治理
95:
96: - 任务
97: - 云函数去除列表接口内的 `getTempFileURL` (改为返回 `cloud://`), 由客户端统一转换 (先灰度分流 10%/30%/100%)。
98: - 引入 “revision 水位线”:
99: - 变更时写入 per-user/全局 revision; 客户端 onShow/interval 对比, 落后时批量失效关键缓存。
100: - 可选: 对用户/未读等小集合使用 CloudBase 实时推送 (watch), 实现秒级刷新。
101: - 交付物
102: - 云函数改造 PR (getPostList, getHotFeed, imageManager 等)。
103: - 灰度/回滚配置与实施文档。
104: - 验收标准
105: - 灰度期间错误率不升高; 调用量与时延指标稳定或下降。
106:
107: ### Phase 5 (P2, 持续): 优化与清理
108:
109: - 任务
110: - 指标看板: 命中率、P75/P95 加载耗时、云函数调用量。
111: - 策略调优: 按业务热度调 TTL/SWR; 冷热分层与黑/白名单键。
112: - 文档与示例保持更新; 清理废弃路径与冗余代码。
113:
114: ## 统一 API 设计 (摘要)
115:
116: ```ts
117: // cache-manager
118: type CacheOptions = { ttlMs?: number; swrMs?: number };
119: type Namespace = {
120: get: (key: string) => Promise<any> | any;
121: set: (key: string, val: any, opts?: CacheOptions) => Promise<void> | void;
122: delete: (key: string) => Promise<void> | void;
123: clear: () => Promise<void> | void;
124: getOrFetch: <T>(key: string, loader: () => Promise<T>, opts?: CacheOptions) => Promise<T>;
125: };
126: namespace (name: string): Namespace;

```
127:
128: // file-url-cache (对外不变)
129: getTempUrls(fileIds: string[]): Promise<Record<string, string>>;
130: getTempUrl(fileId: string): Promise<string>;
131: warm(fileIds: string[]): void;
132: invalidate(ids?: string | string[]): void;
133: setResolver(fn: (ids: string[]) => Promise<Record<string, { url: string; maxAgeSec?: number }>>): void;
134: ```
135:
136: ## 风险与缓解
137:
138: - 旧图/不一致（跨设备）：用版本参数或新路径、缩短 TTL、onShow 轻刷新、revision 机制。
139: - Storage 空间/内存占用：容量上限 + LRU 淘汰；按命名空间清理。
140: - 兼容性（多端差异）：优先 tcb，其次 wx.cloud，最后云函数兜底；统一错误处理与重试。
141: - 监控缺失：上线即埋点与日志开关；设定阈值告警。
142:
143: ## 工期与分工建议（可并行）
144:
145: - Phase 1: 1-2 天（1 前端）
146: - Phase 2: 2-4 天（1 前端）
147: - Phase 3: 3-5 天（1 前端 + 1 后端）
148: - Phase 4: 2-4 天（1 前端 + 1 后端）
149: - Phase 5: 持续（0.5 人天/周巡检）
150:
151: ## 验收指标（关键 KPI）
152:
153: - 云函数 `getTempFileURL` 总调用量下降  $\geq 40\%$ 。
154: - 首屏/翻页图片可见时间 P75 降低  $\geq 20\%$ 。
155: - 列表接口聚合耗时 P75 降低  $\geq 20\%$ （移除 lookup 后）。
156: - 重大问题零回滚（灰度期间）。
157:
158: ——
159:
160: 如同意此计划，我将先提交 Phase 1 的混入/工具与页面接入 PR，然后推进 CacheManager 的实现与替换。
161:
162: ——
163:
164: ## 已完成（2025-10-11）
165:
166: - 通用与工具
167:   - 新增统一缓存工具 CacheManager（_utils/cache-manager.js）：内存 LRU + 可选 uniStorage 持久化，支持 getOrFetch + SWR。
168:   - 临时 URL 缓存接入 CacheManager 命名空间（fileUrls），保留原 API（_utils/file-url-cache.js）。
169:   - 新增 hydrate 工具（_utils/hydrate-temp-urls.js），统一批量映射 cloud://  $\rightarrow$  http(s)。
170:   - 新增全局事件工具（utils/events.js）：EVENTS 常量与 emitAvatarUpdated/emitPostCreated/emitFavoriteChanged。
171:
172: - 页面接入（首批）
173:   - 首页（pages/index/index.vue）：图片 URL hydrate + 预热；未读数改为缓存封装（api-cache/unread.js）。
174:   - 详情页（pages/post-detail/post-detail.vue）：post 与评论图片 hydrate；收藏成功触发 favorite-changed（修复过一次语法…
175:   - 他人主页（pages/user-profile/user-profile.vue）：资料与帖子分页改为缓存（api-cache/user-profile.js），仅监听头像变更…
176:   - 我的主页（pages/profile/profile.vue）：我的帖子与收藏分页改为缓存（api-cache/my.js）；绑定事件驱动失效（头像/发帖/收…
```

177: - 头像编辑页 (pages/profile-edit/profile-edit.vue) : 保存成功触发 avatar-updated。
178: - 发帖页 (pages/add/add.vue) : 发布成功触发 post-created。
179: - 收藏夹选择组件 (components/folder-selector/folder-selector.vue) : 收藏成功触发 favorite-changed (同时保留组件内 favor...
180:
181: - API 缓存封装
182: - api-cache/unread.js: 未读数, TTL 60s + SWR 60s (首页已接入)。
183: - api-cache/profile.js: 我的资料 (持久化, TTL 30min + SWR 5min, 预留给后续页面)。
184: - api-cache/tags.js: 标签列表 (持久化, TTL 30min + SWR 5min, 预留)。
185: - api-cache/user-profile.js: 他人主页的资料与帖子 (无 TTL, 仅事件触发失效)。
186: - api-cache/my.js: 我的资料/帖子/收藏 (无 TTL, 仅事件触发失效)。
187:
188: - 触发与监听规范 (已落地)
189: - 触发点:
190: - 发帖成功 (pages/add/add.vue) → post-created
191: - 头像保存成功 (pages/profile-edit/profile-edit.vue) → avatar-updated
192: - 收藏成功 (components/folder-selector/folder-selector.vue、pages/post-detail/post-detail.vue) → favorite-changed
193: - 取消收藏成功 (pages/profile/profile.vue) → favorite-changed (favored:false)
194: - 监听:
195: - 他人主页仅监听 avatar-updated、post-created
196: - 我的主页监听 avatar-updated、post-created、favorite-changed
197:
198: ---
199:
200: ## 下一步 (建议优先级与粒度)
201:
202: P1 (优先)
203: - 发现页 (index.vue: discover) 与标签筛选页 (pages/tag-filter/tag-filter.vue)
204: - 列表分页接入 CacheManager: namespace 形如 posts:discover:<page>、posts:tag:<tag>:<page> (TTL 60 - 120s + SWR 30 - 60s)
205: - 使用 hydrateTempUrls 与 warm 预热
206: - 首页/发现页聚合降负
207: - 云函数 getPostList/getHotFeed 去除服务端 getTempFileURL 转换 (先灰度), 返回原始 cloud://, 完全由前端缓存解析
208:
209: P1 (结构与一致性)
210: - 收藏夹列表页与收藏夹删除操作
211: - 在 “删除收藏夹” 与 “批量取消收藏” 成功后, emit favorite-changed (可不带 postId, 监听侧做整页失效)
212: - 我的主页 favorites 命名空间增加整页失效入口 (已支持 invalidateMyFavorites() 全清)
213: - 统一 dataCache 双写与迁移
214: - 将历史 dataCache 键映射到 CacheManager (过渡期双写/回读), 最终移除 dataCache 依赖
215:
216: P2 (可选增强)
217: - 引入 revision 水位线
218: - 云端维护 per-user/global revision; 前端 onShow 轻量对比, 落后则批量失效关键缓存
219: - 监控与观测
220: - 埋点命中率/耗时 (命名空间/键维度), 看板化; 发现异常命中或大键及时调整 TTL/SWR
221:
222: ---
223:
224: ## 提交与发布
225:
226: - 当前环境未安装 Git, 已在仓库生成待提交说明:

```
227: - .pending_commits/2025-10-11-commit-01.txt ~ 07.txt (按阶段与功能拆分)
228: - 本机提交命令:
229: - git add -A
230: - git commit -F .pending_commits/2025-10-11-commit-0X.txt (依次提交或 squash)
231:
232:
233: ---
234:
235: ## 追加实施记录 (2025-10-11)
236:
237: 以下为本次在“统一缓存与一致性改造计划”基础上已落地的补充与优化, 便于对照验收与后续迭代。
238:
239: ### 一、列表与页面接入 (P1 优先项)
240:
241: - 首页广场分页缓存 (posts:home)
242: - 新增 `api-cache/home-posts.js` (TTL 90s + SWR 45s)。
243: - `pages/index/index.vue`:
244:   - `getPostList` 改为调用 `getHomePosts`, 保留 `normalize` + `hydrateTempUrls` + 预热; 删除旧的直调 `this.$tcb.callFunc...`
245:   - `getIndexData` 改为直接通过 `getHomePosts` 首屏拉取; 去除 `dataCache` 首屏命中与后续读写。
246:   - 去除点赞状态的 `dataCache` 同步逻辑 (由 `CacheManager` 接管)。
247:
248: - 发现页推荐缓存 (posts:discover)
249: - 新增 `api-cache/discover.js` (TTL 90s + SWR 45s)。
250: - `pages/index/index.vue` 的 `loadRecommendationPosts` 已切换到缓存封装, 并继续执行 `hydrate` + 预热。
251:
252: - 标签筛选分页缓存 (posts:tag:<tag>)
253: - 新增 `api-cache/tag-posts.js` (TTL 90s + SWR 45s)。
254: - `pages/tag-filter/tag-filter.vue` 接入缓存封装, `normalize` + `hydrate` + 预热。
255:
256: ### 二、临时 URL 持久化与失效
257:
258: - `_utils/file-url-cache.js`
259:   - 批量换取结果现在会同步写入 `cacheManager.namespace('fileUrls')` (持久化); `invalidate()` 同步清理命名空间, 冷启动也...
260:   - 仍保留 55min 默认 TTL 与 1min 安全裕量 (SKEW), 列表页继续支持预热 (warm)。
261:
262: ### 三、事件驱动的跨页一致性
263:
264: - 新增 `api-cache/events.js`: 桥接全局事件 → 缓存失效
265:   - `post-created` → 失效首页 (posts:home) 并清理发现页缓存 (posts:discover)。
266:   - `avatar-updated` → 失效用户资料与其主页分页 (profiles:user / userPosts:<uid>)。
267:   - 暂不处理收藏夹整页失效 (按现阶段取舍)。
268: - `main.js`: 在初始化处调用 `setupCacheEventBridges()` 完成一次性注册。
269:
270: ### 四、我的主页体验优化 (头像/签名首屏慢)
271:
272: - `api-cache/my.js`
273:   - `getMyInfo` 调整为 TTL 30min + SWR 5min, 并兼容 `result.userInfo / result.profile / result.data` 等后端返回结构。
274:   - 若 `avatarUrl` 为 `cloud://`, 在 loader 内预先通过 `fileUrlCache.getTempUrl()` 映射为可访问 URL, 并在可用时拼接 `?v=<...`
275:   - 修正工具导入路径为 `@/_utils/*`。
276: - `pages/profile/profile.vue`
```

277: - 新增 `fetchUserProfileFast()`，直接使用 `getMyInfo` (SWR) 返回，替换原来的串行 `getOpenId → getMyProfileData` 请求...

278: - 移除脱离方法体的旧直调代码，修复 Unexpected keyword 'this' 编译错误。

279:

280: ### 五、提交记录

281:

282: - 由于当前环境未装 Git，额外新增 `/.pending\_commits/2025-10-11-commit-08.txt`，说明上述改动内容与建议提交命令。

283:

284: ### 六、当前效果与验证建议

285:

286: - 首页/发现/标签: 90s 内二次进入命中缓存; 90 - 135s 为 SWR 窗口，先展示缓存、后台刷新; 图片 URL 通过 hydrate 转换，避免过...

287: - 我的主页: 资料与头像首屏更快，二次进入几乎秒到; 头像跨设备更新依赖事件桥接与 `?v=` 版本参数。

288:

289: ### 七、后续建议 (可作为下一阶段)

290:

291: 1) 登录完成后的资料与头像预热: 在 `main.js` 登录完成后并行调用 `getMyInfo()` 并对头像做 `fileUrlCache.warm()`，实现“未...

292: 2) CacheManager 观测与开关: 新增 `CACHE\_DEBUG` 与 `hits/misses/evictions` 统计，提供按命名空间打印方法，便于压测与调参。

293: 3) dataCache 全量迁移/清理: 搜索仓库残留使用点，逐一替换为 CacheManager; 首页已完成。

294: 4) 后端灰度“只返回 cloud://”: 列表类接口移除服务端 `getTempFileURL`，10%→30%→100% 渐进; 前端已具备 hydrate 能力。

295: 5) revision 水位线 (可选增强): 轻量对比 per-user/global revision，落后即时批量失效关键缓存。

296:

297: ---

298:

299: ## 使用说明 (新增能力)

300:

301: - 资料与头像预热

302: - 位置: `main.js`

303: - 行为: 登录完成 (检测 `globalData.\_loginProcessCompleted` 且存在 `openid`) 后自动调用 `getMyInfo` 进行预热; 头像在接...

304: - 观察: 控制台出现 `□ [prewarm] getMyInfo done`。

305:

306: - CacheManager 调试与统计

307: - 位置: `@/\_utils/cache-manager.js`、`main.js`

308: - 开关: `uni.setStorageSync('CACHE\_DEBUG', '1')` 后重启; 或运行时调用 `uni.\$cacheDebug(true|false)` 动态切换。

309: - 查看: `uni.\$cacheStats()` 输出各命名空间 `hits/misses/evictions/size` 等统计。

310: - 作用: 辅助压测与监控缓存命中情况，便于调整 TTL/SWR 与容量。

311:

312: ---

313:

314: ## 彻底迁移与清理 (已完成)

315:

316: - dataCache 适配与移除直依赖

317: - 删除 `utils/dataCache.js` (历史适配层已移除); 工程内不再直接依赖 dataCache。

318: - `utils/avatarCache.js` 与 `utils/followCache.js` 已改为直接使用 CacheManager 命名空间，不再依赖 dataCache:

319: - 头像命名空间: `avatars` (TTL 24h, 持久化)。

320: - 关注命名空间: `follow:<currentUserId>` (TTL 1h, 持久化)。

321: - `pages/index/index.vue` 修复预加载依赖，显式导入 `utils/followCache.js`。

322: - `utils/likeService.js`: 点赞后会在 CacheManager 命名空间内进行局部键更新 (`posts:home`、`posts:discover`、`posts:tag:...

323:

324: - 验证与回归项

325: - 首页加载后执行 `uni.\$cacheStats()`，应看到 `avatars` 与 `follow:<uid>` 命中增长。

326: - 清除用户关注缓存: 调用 `require('utils/followCache.js').clearUserFollowCache('<uid>')`，再次进入应触发加载并回写。

327:
328: ---
329:
330: ## 改造后现状 (2025-10-11)
331:
332: 以下为当前“统一缓存与一致性”上线后的实际缓存拓扑、时效策略与触发器，便于全局把握与后续调优。
333:
334: ### 缓存命名空间与时效
335:
336: - 列表/分页 (持久化, 通用 TTL 90s + SWR 45s)
337: - `posts:home`: 广场首页分页。
338: - `posts:discover`: 发现页推荐 (单页)。
339: - `posts:tag:<tag>`: 标签筛选分页。
340:
341: - 资料/详情
342: - `me:info`: 我的资料; TTL 30min + SWR 5min; 持久化; 登录完成会预热 (main.js)。
343: - `me:posts`: 我的帖子; TTL 0 (仅事件失效)。
344: - `me:favorites`: 我的收藏; TTL 0 (仅事件失效)。
345: - `profiles`: 我的资料 (备用封装, TTL 30min + SWR 5min, 持久化)。
346: - `profiles:user`: 他人资料; TTL 0 (仅事件失效)。
347: - `userPosts:<userId>`: 他人主页帖子; TTL 0 (仅事件失效)。
348:
349: - 运营/基础数据
350: - `tags`: 标签列表; TTL 30min + SWR 5min; 持久化。
351: - `unread`: 未读数; TTL 60s + SWR 60s; 非持久化。
352:
353: - 媒体/临时 URL
354: - `fileUrls`: cloud:// → https 临时 URL 映射; 有效期≈55min (预留 1min 裕量); 当前为内存 + 命名空间存取 (非持久化) ...
355:
356: - 头像与关注
357: - `avatars`: 用户头像/昵称/个签; TTL 24h; 持久化; 头像为 cloud:// 时在写缓存前完成一次映射 (fileUrlCache)。
358: - `follow:<currentUserId>`: 关注状态; TTL 1h; 持久化; 支持批量预热。
359:
360: 说明: SWR 表示“过期后在 SWR 窗口内返回陈旧值并后台刷新”，提升首屏可见速度与滚动顺畅度。
361:
362: ### 触发器 (事件/动作)
363:
364: - 全局事件 (见 `api-cache/events.js` + `main.js`)
365: - `post-created`: 失效 `posts:home`, 清理 `posts:discover`。
366: - `avatar-updated`: 失效 `me:info` 与 `profiles:user` / `userPosts:<uid>` (若在他人主页)。
367: - `favorite-changed`: 当前阶段按你的要求不做整页失效 (留给后续增强)。
368:
369: - 页面内动作
370: - 点赞成功: 在 CacheManager 的 `posts:home` / `posts:discover` / `posts:tag:\*` / `me:posts` / `userPosts:\*` 中“局部键...
371: - 临时 URL 相关: 上传/替换/删除成功后对涉及的 fileId 调用 `fileUrlCache.invalidate()`; 页面 hydrate 负责批量换取与预热。
372: - 登录完成: `main.js` 自动预热 `me:info` (首进“我的”更快)。
373:
374: ### 调试与观测
375:
376: - 打开/关闭调试: `uni.\$cacheDebug(true|false)`。

377: - 查看统计: `uni.\$cacheStats()` (输出各命名空间的 `hits/misses/evictions/gets/sets/size`)。
378:
379: ---
380:
381: ## 可继续优化方向 (建议)
382:
383: - 收藏变更治理 (P1)
384: - 收藏/取消收藏/删除收藏夹后, 按页或整页对 `me:favorites`、`posts:home|discover|tag:\*` 进行“局部键更新”或快速失效, ...
385:
386: - 后端“只返回 cloud://”灰度 (P1)
387: - 去除列表类接口中的 `getTempFileURL`, 分 10% → 30% → 100% 灰度; 依赖前端统一 `hydrate`; 观测云函数调用量与 P75 指标...
388:
389: - revision 水位线 (P2)
390: - 维护 per-user / global revision; 前端在 onShow 或轮询对比, 落后时批量失效关键命名空间 (如 me:info、posts:*、tags 等...
391:
392: - 更新范围收敛 (P2)
393: - 将点赞/收藏后的局部更新限定到“当前活跃分页附近”的键, 减少不必要的扫描与更新。
394:
395: - 可视化看板 (P2)
396: - 将 `uni.\$cacheStats()` 序列化上报到看板 (如云函数/日志服务), 长期跟踪命中率与流量下降幅度。

===== FILE: 重构指南.md =====

- 1: 适用于所有页面的通用重构SOP (Standard Operating Procedure)
- 2: 核心哲学: 先解耦, 后拆分。每一步都进行验证。
- 3: 对于您项目中的任何一个 .vue 页面, 请严格遵循以下步骤:
- 4: 阶段一: 分析与准备 (耗时10%)
- 5: 在动手修改任何代码之前, 先做好侦察工作。
- 6: 选择目标页面: 选择您下一个要重构的页面 (例如 post-detail.vue)。
- 7: 分析 <script>:
- 8: 识别API调用: 找出所有 this.\$tcb.callFunction(...) 的地方。问自己: “这是在操作哪个资源 (帖子、评论、用户)?”
- 9: 识别工具函数: 找出所有可以脱离组件存在的纯函数 (如数据格式化、URL处理、复杂计算等)。
- 10: 识别页面逻辑: 剩下的就是只与本页面强相关的逻辑 (如弹窗的显示/隐藏、动画控制等)。
- 11: 分析 <template>:
- 12: 寻找“块”: 找出页面上独立的、有明确边界的UI区域。例如: 帖子头部、作者信息卡片、评论区、评论项、相关推荐列表等。这些都是...
- 13: 寻找“循环”: 任何 v-for 循环内部的内容, 几乎都是一个完美的组件拆分对象 (例如评论列表中的每一个评论项)。
- 14: 阶段二: 执行重构 (耗时80%)
- 15: 这是流水线的核心工序, 严格分为“逻辑”和“视图”两步。
- 16: A. 逻辑层解耦
- 17: 迁移API调用:
- 18: 打开 src/api-cache/ 目录。
- 19: 根据您在步骤3中识别的资源, 找到对应的文件 (如 post.js, comment.js), 如果没有就创建一个。
- 20: 将被识别出的API调用封装成独立的、语义化的函数 (如 getPostDetail(postId), postComment(data)), 并 export。
- 21: 关键: 在函数内部, 使用您项目已有的 utils/cloudCall.js。
- 22: 迁移工具函数:
- 23: 打开 src/utils/ 目录。
- 24: 将被识别出的工具函数, 放入对应的文件中 (如 time.js, formatters.js), 并 export。
- 25: 替换并验证:
- 26: 回到目标页面, 在 <script> 顶部 import 您刚刚创建的API函数和工具函数。
- 27: 用新函数替换掉页面中原有的内联逻辑。
- 28:

- 29: B. 视图层拆分
- 30: 创建子组件:
- 31: 根据您在步骤4中识别出的UI“块”和“循环项”，在 src/components/ 目录下创建新的 .vue 组件（例如 CommentItem.vue, AuthorCar...
- 32: 定义“组件合同”（Props & Events）:
- 33: Props（数据向下流动）：在新组件中，明确定义它需要从父组件接收哪些数据。
- 34: 在父组件中实现:
- 35: 在目标页面（父组件）的 <template> 中，删除旧的大块HTML，替换为新的组件标签（<CommentItem :comment="item" @reply="handleRe...
- 36: 在 <script> 中 import 并注册新组件，并实现监听事件的方法（handleReply）。
- 37: 迭代与验证:
- 38: 重复步骤8-10，直到您觉得页面 <template> 已经足够清晰为止。
- 39: 阶段三：收尾与合并（耗时10%）
- 40: 最终测试：对整个页面进行一次完整的功能回归测试，确保所有功能（包括各种边缘情况）都正常。
- 41: 代码清理：删除被注释掉的旧代码，检查新代码的格式和命名规范。

===== FILE: 项目开发总结.md =====

```
1: # 回车键项目开发总结
2:
3: ## □ 项目概述
4:
5: ### 技术栈
6: - **前端框架**: uni-app (Vue 2)
7: - **云服务**: 腾讯云开发 (wx-server-sdk) + CloudBase SDK (@cloudbase/js-sdk)
8: - **数据库**: 云数据库，统一使用 `_openid` 字段存储用户标识
9: - **云存储**: CloudBase 云存储
10: - **多平台支持**: 小程序、App、H5
11:
12: ### 项目特点
13: - 小程序转uni-app项目
14: - 多平台兼容性要求高
15: - H5和app使用CloudBase SDK，小程序使用wx-sdk
16:
17:
18: ### 2. 腾讯云开发环境配置
19:
20: #### 环境信息
21: - **环境ID**: `cloud1-5gb0pbyl400845f5`
22: - **配置文件**: `cloudbaserc.json`
23: - **框架类型**: uni-app
24: - **运行时**: Node.js 16.13
25:
26: #### 云函数调用方式
27: ```javascript
28: // □ 正确方式 (TCB)
29: if (this.$tcb && this.$tcb.callFunction) {
30:   this.$tcb.callFunction({
31:     name: 'getUserProfile',
32:     data: { userId: authorOpenid }
33:   }).then((res) => {
34:     // 处理成功响应
35:   }).catch((err) => {
```

```

36:         // 处理错误
37:     });
38: }
39: ```
40:
41: ##### 数据库访问兼容性
42: ```javascript
43: // 兼容性数据库访问
44: const getDatabase = () => {
45:     if (this.$tcb && this.$tcb.database) {
46:         return this.$tcb.database(); // H5环境
47:     } else if (typeof wx !== 'undefined' && wx.cloud && wx.cloud.database) {
48:         return wx.cloud.database(); // 小程序环境
49:     } else {
50:         throw new Error('无法获取数据库实例');
51:     }
52: };
53: ```
54:
55: ### 3. 环境检测问题
56:
57: ##### 问题描述
58: - 安卓原生端被错误识别为小程序环境
59: - 导致云函数调用失败
60:
61: ##### 解决方案
62: ```javascript
63: // 使用严格的检测逻辑
64: if (typeof wx !== 'undefined' && wx.getSystemInfoSync && wx.getAccountInfoSync) {
65:     try {
66:         const systemInfo = wx.getSystemInfoSync();
67:         const accountInfo = wx.getAccountInfoSync();
68:         if (accountInfo.miniProgram && systemInfo.platform) {
69:             return 'mp-weixin'; // 真正的小程序环境
70:         }
71:     } catch (e) {
72:         // 不是小程序环境
73:     }
74: }
75: ```
76:
77: ### 4. 图片上传和数据库写入问题
78:
79: ##### 问题描述
80: - 图片上传成功但fileID为undefined
81: - 帖子创建成功但图片URL字段缺失
82: - 刚进入应用时显示"用户未登录"提示
83:
84: ##### 解决方案
85: **图片上传云函数返回格式处理**:
```

```

86: ```javascript
87: // □ 正确处理云函数返回结果
88: this.callCloudFunction('upload', {...}).then((uploadRes) => {
89:     if (uploadRes && uploadRes.result && uploadRes.result.success) {
90:         resolve({
91:             fileId: uploadRes.result.fileId,
92:             cloudPath: uploadRes.result.cloudPath
93:         });
94:     } else {
95:         reject(new Error('上传云函数返回格式错误'));
96:     }
97: });
98: ```
99:
100: **数据库字段完整性**:
101: ```javascript
102: // □ 确保包含所有图片URL字段
103: if (validFileIds.length > 0) {
104:     postData.imageUrl = validFileIds[0];
105:     postData.imageUrls = validFileIds;
106:     postData.originalImageUrl = validOriginalFileIds[0];
107:     postData.originalImageUrls = validOriginalFileIds;
108: }
109: ```
110:
111: ### 5. 登录状态管理问题
112:
113: #### 问题描述
114: - 缓存登录成功后, 页面仍显示未登录
115: - globalData设置方式不正确
116:
117: #### 解决方案
118: ```javascript
119: // 确保 getApp().globalData 也被正确设置
120: const appInstance = getApp();
121: if (appInstance) {
122:     appInstance.globalData = appInstance.globalData || {};
123:     appInstance.globalData.userInfo = cachedUserInfo;
124:     appInstance.globalData.openid = cachedUserInfo._openid;
125: }
126: ```
127:
128: **登录提示优化**:
129: ```javascript
130: // □ 避免应用启动初期显示登录提示
131: const isAppStarting = !appInstance.globalData || !appInstance.globalData._loginProcessCompleted;
132: if (!isAppStarting) {
133:     uni.showToast({ title: '用户未登录', icon: 'none' });
134: }
135: ```

```

```
136:
137: ### 6. 文件上传问题
138:
139: #### H5环境特殊性
140: - `uni.chooseImage` 返回的 `res.tempFiles[0]` 是File对象
141: - blob URL无法直接用于CloudBase上传
142:
143: #### 解决方案
144: ```javascript
145: // 通过云函数处理文件上传
146: const reader = new FileReader();
147: reader.readAsDataURL(fileObject);
148: reader.onload = () => {
149:   const base64 = reader.result.split(',')[1];
150:   this.$tcb.callFunction({
151:     name: 'uploadAvatar',
152:     data: { fileData: base64, cloudPath, openid }
153:   });
154: };
155: ```
156:
157: **兼容性文件上传方法**:
158: ```javascript
159: // □ 推荐方式
160: this.uploadFile(cloudPath, filePath).then((res) => {
161:   // 处理上传成功
162: }).catch((err) => {
163:   // 处理上传失败
164: });
165: ```
166:
167: ### 7. 图片显示问题
168:
169: #### 问题描述
170: - `cloud://` 协议在浏览器中无法直接访问
171: - 需要获取临时URL
172:
173: #### 解决方案
174: ```javascript
175: // 使用CloudBase SDK获取临时URL
176: this.$tcb.getTempFileURL({
177:   fileList: [fileID]
178: }).then((tempRes) => {
179:   const tempUrl = tempRes.fileList[0].tempFileURL;
180:   this.setData({
181:     localAvatarTempPath: tempUrl
182:   });
183: });
184: ```
185:
```

```
186: ## □ 登录系统重构
187:
188: ### 主要修改内容
189:
190: 1. **开屏页面跳转逻辑修改**
191:   - 检测不到缓存时跳转到登录页面（而非注册页面）
192:
193: 2. **登录页面重构**
194:   - 从头像和昵称设置页面改为Poem ID和密码登录页面
195:   - 新增Poem ID输入框、密码输入框、登录按钮
196:   - 采用现代化渐变背景和卡片式设计
197:
198: 3. **新增注册页面**
199:   - 支持用户在其他端新建账号
200:   - 包含Poem ID、密码、确认密码、昵称字段
201:   - 支持密码一致性检查和Poem ID唯一性检查
202:
203: 4. **用户表结构扩展**
204:   - 新增 `poemId` 和 `password` 字段
205:
206: 5. **新增云函数**
207:   - `loginWithCredentials`: 根据Poem ID和密码验证用户身份
208:   - `registerUser`: 创建新用户账号
209:
210: ### 多平台兼容性
211: - **H5环境**: 使用TCB SDK (`this.$tcb.callFunction`)
212: - **小程序环境**: 使用微信云开发 (`wx.cloud.callFunction`)
213: - **App环境**: 使用TCB SDK (`this.$tcb.callFunction`)
214:
215: ## □□ 关键修复点
216:
217: ### 1. 云函数兼容性修复
218: ```javascript
219: // 支持多平台
220: const openid = wxContext.OPENID || event.openid || context.OPENID;
221: ```
222:
223: ### 2. 数据库查询统一化
224: ```javascript
225: // 所有云函数都使用相同的查询模式
226: const userRecord = await db.collection('users').where({
227:   _openid: openid
228: }).get();
229: ```
230:
231: ### 3. 模板显示逻辑优化
232: ```html
233: <!-- 优先使用可访问的URL -->
234: <image :src="localAvatarTempPath || avatarFileID"></image>
235: ```
```

```
236:
237: ## □ 项目最佳实践
238:
239: ### 1. 数据库设计原则
240: - 统一使用 `_openid` 字段存储用户标识
241: - 所有云函数都使用相同的查询模式
242: - 保持数据一致性
243:
244: ### 2. 云函数调用检查
245: 在调用云函数前，始终检查TCB实例是否可用：
246: ```javascript
247: if (!this.$tcdb || !this.$tcdb.callFunction) {
248:   console.error('TCB实例不可用');
249:   return;
250: }
251: ```
252:
253: ### 3. 数据库操作检查
254: 在访问数据库前，检查数据库实例：
255: ```javascript
256: if (!this.$tcdb || !this.$tcdb.database) {
257:   console.error('数据库实例不可用');
258:   return;
259: }
260: ```
261:
262: ### 4. 环境兼容性
263: ```javascript
264: // 检查运行环境，使用对应的API
265: const isH5 = typeof window !== 'undefined';
266: const isMiniProgram = typeof wx !== 'undefined';
267:
268: if (isH5) {
269:   // H5环境使用TCB
270:   const db = this.$tcdb.database();
271: } else if (isMiniProgram) {
272:   // 小程序环境使用微信云开发
273:   const db = wx.cloud.database();
274: }
275: ```
276:
277: ### 5. 错误处理策略
278: ```javascript
279: // 云函数调用模式
280: this.callCloudFunction('functionName', {
281:   param1: value1,
282:   param2: value2
283: }).then((res) => {
284:   // 处理成功响应
285: }).catch((err) => {
```

```
286:     console.error('操作失败:', err);
287:     uni.showToast({
288:         title: '操作失败',
289:         icon: 'none'
290:     });
291: }).finally(() => {
292:     // 清理工作
293: });
294: ```
295:
296: ### 6. 云函数部署
297: ```bash
298: # 部署所有云函数
299: cloudbase framework:deploy
300:
301: # 部署单个云函数
302: cloudbase fn deploy [functionName]
303:
304: # 查看云函数列表
305: cloudbase fn list
306:
307: # 查看环境信息
308: cloudbase env list
309: ```
310:
311: ## □ 调试技巧
312:
313: ### 1. 数据流追踪
314: ```javascript
315: // 关键节点添加日志
316: console.log('□ [步骤] 输入数据:', input);
317: console.log('□ [步骤] 处理结果:', result);
318: console.log('□ [步骤] 最终输出:', output);
319: ```
320:
321: ### 2. 问题定位方法
322: 1. 从错误信息入手，分析根本原因
323: 2. 检查SDK版本和API兼容性
324: 3. 验证网络请求和响应格式
325: 4. 使用监控机制追踪数据变化
326:
327: ### 3. 分步骤验证
328: - 先验证基础功能（登录、认证）
329: - 再验证复杂功能（文件上传、图片显示）
330: - 最后验证完整流程
331:
332: ## □ 常见问题快速排查
333:
334: ### 1. TCB实例未定义
335: **症状**: `this.$tcb is undefined`
```

336: ****解决****: 检查main.js中的TCB初始化代码是否正确执行

337:

338: **### 2. 云函数调用失败**

339: ****症状****: 云函数返回错误或超时

340: ****解决****:

341: - 检查云函数是否已部署

342: - 检查环境ID是否正确

343: - 检查网络连接

344:

345: **### 3. 数据库访问失败**

346: ****症状****: 数据库操作返回错误

347: ****解决****:

348: - 检查数据库权限

349: - 检查集合名称是否正确

350: - 检查查询条件是否正确

351:

352: **### 4. 图片上传失败**

353: ****症状****: 上传成功但fileID为undefined

354: ****解决****: 检查云函数返回格式处理逻辑

355:

356: **### 5. 登录状态问题**

357: ****症状****: 应用启动时显示"用户未登录"

358: ****解决****: 添加登录流程完成标志

359:

360: **### 6. Vue组件加载失败**

361: ****症状****: `Failed to resolve async component`

362: ****解决****:

363: - 检查组件导入路径

364: - 检查组件是否存在

365: - 重启开发服务器

366:

367: **## □ 未来优化建议**

368:

369: **### 1. 架构优化**

370: - 考虑完全迁移到CloudBase SDK

371: - 统一技术栈, 减少兼容性问题

372: - 使用云函数作为中间层, 处理平台差异

373:

374: **### 2. 性能优化**

375: - 添加临时URL缓存, 减少API调用

376: - 图片压缩和懒加载

377: - 错误监控和性能监控

378:

379: **### 3. 开发体验优化**

380: - 集成错误监控系统

381: - 添加开发环境调试工具

382: - 完善文档和注释

383:

384: **## □ 经验总结**

385:

386: ### 关键教训

387: 1. **避免SDK混用**: 保持技术栈一致性

388: 2. **重视多平台兼容**: 不同平台有不同的API和限制

389: 3. **完善错误处理**: 提供详细的调试信息和降级方案

390: 4. **分步骤验证**: 复杂问题需要逐步排查

391:

392: ### 调试心得

393: 1. 从错误信息入手, 不要被表面现象迷惑

394: 2. 使用监控机制追踪数据变化

395: 3. 分步骤验证每个环节

396: 4. 保持耐心, 复杂问题需要时间解决

397:

398: ### 架构设计原则

399: 1. 统一数据模型和查询方式

400: 2. 使用云函数处理平台差异

401: 3. 设计统一的错误处理机制

402: 4. 保持代码的可维护性和可扩展性

403:

404: ---

405:

406: *本文档整合了回车键项目开发过程中的关键问题、解决方案和经验总结, 为类似项目提供参考。*